SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Metódy Nasadzovania Python Aplikácií Pomocou Generatívnej Umelej Inteligencie

Minimová Práca

2026 **Ing. Rastislav Tvarožek**

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Metódy Nasadzovania Python Aplikácií Pomocou Generatívnej Umelej Inteligencie

Minimová Práca

Študijný program:	Mechatronické systémy
Študijný odbor:	Kybernetika
Školiace pracovisko:	Ústav automobilovej mechatroniky
Školiteľ:	doc. Ing. Oto Haffner, PhD.

2026 **Ing. Rastislav Tvarožek**

Abstrakt

Kľúčové slová

GenAI, LLM, Python

Abstract

Keywords

GenAI, LLM, Python

Obsah

Úvod **9**

1 Vymedzenie pojmov **11**
1.1 Nasadzovanie do produkcie 11
1.2 Typy nasadzovaných aplikácií 13
1.2.1 Webové aplikácie 13
1.2.2 Aplikačné programové rozhrania 15
1.2.3 Automatizačné skripty 19
1.3 Použitá sada nástrojov pre nasadzovanie aplikácií 21
1.3.1 Python správa prostredia 22
1.3.2 Kontajner a kontajnerizácia aplikácií 23
1.3.3 Orchestrácia kontajnerov 24
1.3.4 Proces nasadenia kontajnerizovanej aplikácie 25
1.4 Generatívna umelá inteligencia 26
1.4.1 AI Agent 26
1.4.2 LLM nástroje 27
1.4.3 Multi-agent Systém 28
1.4.3.1 Vnorení agenti 29
1.4.3.2 Mechanizmus odovzdávania 29
1.4.3.3 Mechanizmus dynamických zručností 31
1.4.3.4 Smerovač 31

2 Súčasný stav **33**
2.1 PaaS 33
2.2 LLM služby 35
2.3 Zhodnotenie aktuálneho výskumu danej problematiky 36

3 Motivácia a ciele práce **37**

4 Návrhy riešenia pomocou generatívnej umelej inteligencie **39**
4.1 Riadený proces využívajúci LLM 39
4.2 Monolitický agent 42
4.3 Multi-agentický systém využívajúci vnorených agentov 46

5 Overenie konceptu **49**
5.1 Experimentálna metodológia 49
5.1.1 Cieľ experimentov 49

5.1.2 Testovacie vstupy 49
5.1.3 Postup experimentov 50
5.1.4 Zaznamenávané metriky 51
5.1.5 Spôsob vyhodnocovania 55
5.2 Experimentálne prostredie 56
5.2.1 Hardvérové prostredie 56
5.2.2 Softvérové prostredie 57
5.2.3 Špecifikácia použitého LLM 57
5.3 Experimenty a preukázanie konceptu 57
5.3.1 Riadený proces 58
5.3.2 Monolitický agent 58
5.3.3 Multi-agentický Systém 58
5.3.4 Porovnanie výsledkov 58

Záver **59**

Literatúra **60**
Použitie nástrojov umelej inteligencie 62

Zoznam obrázkov a tabuliek

Obrázok 1 Zjednodušená vizualizácia životného cyklu softvérového systému 12
Obrázok 2 Vizualizácia REST API architektúry 17
Obrázok 3 Vizualizácia Multi-agent architektúry využívajúcej vnorených agentov 29
Obrázok 4 Vizualizácia Multi-agent architektúry využívajúcej mechanizmus odovzdávania 30
Obrázok 5 Vizualizácia Multi-agent architektúry využívajúcej mechanizmus dynamických zručností 31
Obrázok 6 Vizualizácia Multi-agent architektúry využívajúcej smerovač 32
Obrázok 7 Vizualizácia návrhu riadeného procesu využívajúceho LLM 40
Obrázok 8 Vizualizácia monolitického agenta 43
Obrázok 9 Vizualizácia Multi-agentického Systému 46

Tabuľka 1 Porovnanie PaaS s funkciou nasadenia zo zdrojového kódu 34
Tabuľka 2 Porovnanie ceny PaaS schopných nasadenia zo zdrojového kódu 34
Tabuľka 3 Prehľad testovacích Python aplikácií použitých v experimentoch 50
Tabuľka 4 Prehľad testovaných aspektov pomocou jednotlivých testovacích aplikácií 51

Zoznam značiek a skratiek

CI/CD – kontinuálna integrácia a kontinuálne doručovanie, z angl. *Continuous Integration/Continuous Delivery*
API – aplikačné programové rozhranie, z angl. *Application Programming Interface*
WSGI – *Web Server Gateway Interface*
ASGI – *Asynchronous Server Gateway Interface*
URI – jednotný identifikátor prostriedku z angl. *Uniform Resource Identifier*
LLM – veľké jazykové modely, z angl. *Large Language Models*
PaaS – platforma ako služba, z angl. *Platform as a Service*
CNCF – *Cloud Native Computing Foundation*
CLI – rozhranie príkazového riadku, z angl. *Command Line Interface*
PyPI – *Python Package Index*
RPC – *Remote Procedure Call*

Zoznam výpisov kódov

1 Ukážka jednoduchej Flask aplikácie 15
2 Ukážka GET požiadavky v jazyku Python 17
3 Inštalácia FastAPI a ASGI servera uvicorn 19
4 Príklad použitia FastAPI 19
5 Príklad dátového toku 20
6 Príklad definície LLM nástroja 27

Úvod

Súčasná etapa priemyselného technologického vývoja, označovaná ako **Industry 4.0**, je charakteristická digitalizáciou, automatizáciou a prepájaním kyberneticko-fyzikálnych systémov. Jedným zo základných princípov tohto konceptu je **modularita**, ktorá umožňuje rozdelenie komplexných systémov na menšie, nezávislé časti, ktoré je možné flexibilne nasadzovať, upravovať a opätovne používať. V oblasti softvérového inžinierstva sa tento princíp realizuje prostredníctvom distribuovaných aplikácií a využitia technológií kontajnerizácie, ktoré zabezpečujú izoláciu aplikácií a ich prenositeľnosť. Súčasne medzi najpoužívanejšie technológie patrí Docker, ktorý umožňuje distribuovať aplikáciu spolu so všetkými jej závislosťami do jednotného celku a zabezpečiť jej konzistentnú funkcionalitu naprieč rôznymi prostrediami.

Hlavným cieľom predloženej minimovej práce je objasnenie problematiky nasadzovania počítačových aplikácií do produkčných prostredí. Produkčné prostredia predstavujú prostredia, v ktorých sú tieto aplikácie priamo využívané koncovými používateľmi. Súčasťou práce je predstavenie možného nového riešenia uvedeného problému, ktoré využíva generatívnu umelú inteligenciu.

Nasadenie aplikácie do takzvaného produkčného prostredia predstavuje jeden z posledných krokov vývojového procesu, pokiaľ neberieme do úvahy následné fázy testovania, monitorovania a ďalšie kroky vyplývajúce z výsledkov týchto činností. Tento proces je pre vývojárov zameraných predovšetkým na aplikačnú logiku často málo známy alebo zvládnutý len na nevyhnutnej úrovni. V dôsledku toho môže byť samotné nasadzovanie aplikácie časovo náročnejšie, než je potrebné, a vývojár je nútený venovať svoj čas aktivitám, ktoré len okrajovo súvisia s jeho primárnou rolou.

Z tohto dôvodu vznikajú špecializované pracovné pozície a profesijné zamerania orientované najmä na proces nasadzovania aplikácií do rôznych prostredií. Tieto pozície sú zvyčajne zastrešené takzvanými operačnými tímami, prípadne môžu byť v menších projektoch vykonávané jednotlivcom. Cieľom operačných tímov je zabezpečiť čo najrýchlejší a najspoľahlivejší presun aplikácie z vývojového prostredia do produkcie a zároveň umožniť opakovanie tohto procesu podľa potrieb vyplývajúcich z monitorovania a testovania aplikácie. Súčasne sú tieto tímy zodpovedné aj za zabezpečenie bezpečnosti aplikácie v produkčnom prostredí.

Proces nasadzovania aplikácií sa líši v závislosti od typu danej aplikácie, ako aj od technológií použitých na jej nasadenie. Táto práca sa zaoberá nasadzovaním aplikácií naprogramovaných v programovacom jazyku Python, ktorý patrí medzi najpoužívanejšie programovacie jazyky a to najmä vďaka svojej jednoduchosti a

širokým možnostiam využitia. Aj v prípade jazyka Python však rozlišujeme viacero typov aplikácií, ako sú napríklad aplikačné programové rozhrania, modely strojového učenia alebo procesy na transformáciu dát, pri ktorých je potrebné zabezpečiť ich opakované spúšťanie v stanovenom čase. Z hľadiska technológií použitých na nasadzovanie sa táto práca opiera o súbor nástrojov a technológií, ktoré patria medzi najrozšírenejšie v praxi, vrátane technológií kontajnerizácie, ako je Docker.

Ako je možné pozorovať, proces nasadzovania umožňuje využitie veľkého množstva technológií a postupov, ktoré spolu vytvárajú komplexnú sieť možných kombinácií. Tento problém je pre operačného inžiniera vo väčšine prípadov riešiteľný, avšak často za cenu značnej časovej náročnosti. Samotné nasadenie, najmä v prípade nového projektu, môže byť časovo náročné aj vtedy, keď má operačný inžinier jednotlivé kroky pre daný typ aplikácie už otestované. Jednou z možností riešenia tohoto problému môže byť využitie komerčných služieb na nasadzovanie aplikácií, do ktorých je možné vložiť zdrojový kód a služba následne poskytne už nasadenú aplikáciu, napríklad platformy typu Google Cloud. Tieto služby sú však spravidla spoplatnené a často optimalizované len pre špecifické prípady použitia.

Na základe vyššie uvedených skutočností vznikla táto minimová práca, ktorej cieľom je riešiť uvedenú problematiku prostredníctvom nového a finančne nenáročného riešenia v podobe aplikácie, ktorá zabezpečuje proces nasadzovania pomocou generatívnej umelej inteligencie.

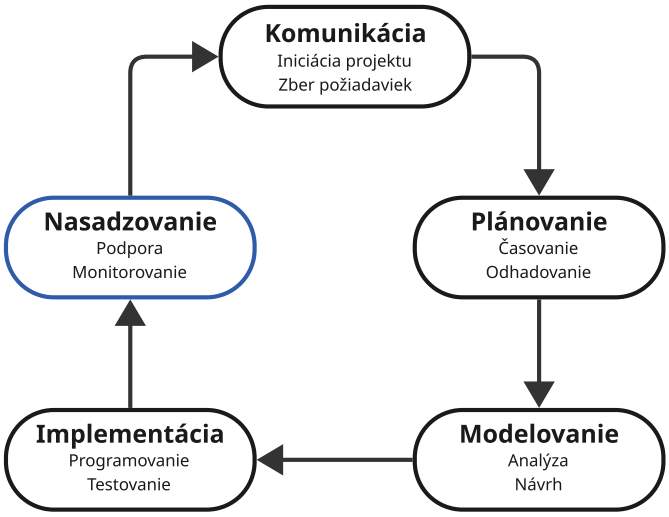
1 Vymedzenie pojmov

Vzhľadom na interdisciplinárny charakter riešenej problematiky, ktorý zahŕňa oblasti vývoja softvéru, produkcionalizáciu a jej náležité technológie a generatívnu umelú inteligenciu, je jednoznačné definovanie kľúčových pojmov nevyhnutné pre správnu interpretáciu obsahu práce. Uvedené definície zároveň tvoria teoretický základ pre ďalšie kapitoly a umožňujú konzistentné používanie odbornej terminológie v celom texte.

1.1 Nasadzovanie do produkcie

Nasadzovanie softvéru do produkcie je možné definovať ako proces, prostredníctvom ktorého je vyvíjaný softvér sprístupnený koncovým používateľom alebo iným softvérovým systémom v produkčnom prostredí. Produkčné prostredie predstavuje reálne prevádzkové prostredie, v ktorom je softvér vystavený skutočnej záťaži, bezpečnostným rizikám a požiadavkám na vysokú dostupnosť a spoľahlivosť [1].

Nasadzovanie do produkcie nadväzuje na fázy implementácie a testovania softvéru a tvorí neoddeliteľnú súčasť životného cyklu softvérového systému. Jeho cieľom nie je len samotné umiestnenie aplikácie do produkčného prostredia, ale aj zabezpečenie správnej konfigurácie, integrácie s existujúcimi systémami a pripravenosti na dlhodobú prevádzku [2]. Pri životnom cykle softvérového systému je možné identifikovať viacero modelov, ktoré tento cyklus popisujú a určujú poradie jeho jednotlivých krokov. Tieto kroky sa však do veľkej miery prekrývajú medzi jednotlivými modelmi a líšia sa najmä ich postupnosťou, frekvenciou, usporiadaním iterácií a podobnými charakteristikami. Vo väčšine prípadov je samotné nasadenie do produkcie poslednou fázou, ktorú je potrebné vykonať v rámci danej iterácie, ako je znázornené na nasledujúcom diagrame 1. Každý z blokov diagramu zobrazuje jednu fázu vývoja softvérového systému, pod každým blokom sa nachádza výber možných častí danej fázy.



Obr. 1: Zjednodušená vizualizácia životného cyklu softvérového systému

Z hľadiska spôsobu realizácie možno rozlišovať viacero typov nasadzovania. Tradičný prístup spočíva v manuálnom nasadzovaní, pri ktorom sú jednotlivé kroky vykonávané ručne. Tento spôsob je však náchylný na chyby a je menej vhodný pre komplexné alebo často aktualizované systémy. Alternatívou je automatizované nasadzovanie, ktoré využíva skripty a nástroje na opakovateľné a konzistentné doručovanie softvéru do produkcie, čím sa znižuje riziko ľudskej chyby a skracuje čas potrebný na uvedenie aplikácie do prevádzky [2].

Takéto automatizované nasadzovanie softvéru do produkcie je úzko späté s princípmi nazývanými CI/CD, Kontinuálna Integrácia a Kontinuálne Doručovanie, z angl. *Continuous Integration/Continuous Delivery*. V rámci tohto prístupu sú jednotlivé kroky procesu, ako je zostavenie aplikácie, testovanie a jej nasadenie do produkčného prostredia, realizované automatizovane prostredníctvom nástrojov a preddefinovaných procesov. Cieľom CI/CD je zabezpečiť, aby zmeny v zdrojovom kóde mohli byť často, spoľahlivo a s minimálnym rizikom doručené do produkčného prostredia, čím sa skracuje čas medzi vývojom a reálnym nasadením softvéru [2].

Tento prístup prispieva k zvýšeniu kvality softvéru, keďže každá zmena je pred nasadením overená automatizovanými testami, a zároveň znižuje riziko chýb spôsobených manuálnym zásahom. V kontexte CI/CD sa nasadzovanie do produkcie stáva opakovateľným a predvídateľným procesom, ktorý je možné efektívne aplikovať aj pri komplexných a často aktualizovaných softvérových systémoch [2].

Teda v prenesení povinností na operačného inžiniera, ktorý je zodpovedný za samotné nasadenie do produkcie, je najskôr potrebné manuálne nasadiť aplikáciu a

následne zostrojiť CI/CD, ktoré bude schopné automaticky vykonávať nasadenie do produkcie.

V praxi sa využívajú aj pokročilé prístupy k nasadzovaniu, ako napríklad postupné(inkrementálne) nasadzovanie alebo paralelné nasadzovanie viacerých verzií aplikácie. Tieto prístupy umožňujú minimalizovať výpadky systému a znižovať dopad prípadných chýb na koncových používateľov. Dôležitou súčasťou nasadzovania je aj možnosť návratu na predchádzajúcu stabilnú verziu systému v prípade zlyhania novej verzie [2].

Moderné prístupy k vývoju softvéru kladú dôraz na prepojenie nasadzovania s kontinuálnou integráciou a kontinuálnym doručovaním, ktoré umožňujú časté kontrolované vydávanie nových verzií softvéru do produkčného prostredia. Tieto prístupy prispievajú k vyššej kvalite softvéru a lepšej reakcii na meniace sa požiadavky používateľov produkčného prostredia [2].

Z dôvodu obmedzeného rozsahu tejto práce sa výskum zaoberá výhradne manuálnym nasadzovaním, v ktorom generatívna umelá inteligencia zastáva pozíciu operačného inžiniera, a výsledkom je nasadená aplikácia, nie CI/CD systém. Širšie zadefinovanie cieľov na základe vyššie uvedených skutočností sa nachádza v jednej z nasledujúcich kapitol.

1.2 Typy nasadzovaných aplikácií

Ako je možné vyvodiť z názvu tejto práce, riešenie problematiky sa zameriava výlučne na aplikácie napísané pomocou programovacieho jazyka Python. Python umožňuje vývoj rôznych typov aplikácií a to od webových, desktopových až po aplikácie realizujúce strojové učenie. Avšak nie všetky tieto aplikácie sú vhodnými adeptmi na produkčné nasadzovanie ako je ponímané v tejto práci na základe nasledujúcej kapitoly. Z tohto dôvodu sa práca zameriava iba na nasledujúce typy Python programov, respektíve aplikácií:

1. webová aplikácia,
2. API, aplikačné programové rozhranie, z angl. *Application Programming Interface*,
3. automatizačné skripty.

1.2.1 Webové aplikácie

Webová aplikácia predstavuje softvérový systém, ktorý je sprístupnený používateľovi prostredníctvom webového prehliadača a nevyžaduje lokálnu inštaláciu. Na rozdiel

od tradičných desktopových aplikácií umožňuje webová aplikácia interaktívnu prácu s dátami a funkciami priamo v prehliadači, pričom všetky výpočty a spracovanie logiky prebiehajú na vzdialenom serveri [3].

Webové aplikácie nachádzajú široké uplatnenie v rôznych oblastiach, vrátane elektronického obchodu, online bankovníctva, správy e-mailov alebo tímovej spolupráce, kde poskytujú používateľom možnosť vykonávať interaktívne úlohy prostredníctvom formulárov, ovládacích prvkov a dynamického obsahu [3].

Technologicky sú webové aplikácie založené na klient-server architektúre, v ktorej klientská časť zobrazuje používateľské rozhranie a odosiela požiadavky na server, zatiaľ čo serverová časť spracováva tieto požiadavky, vykonáva aplikačnú logiku a zabezpečuje prístup k dátam [3].

Významnou prednosťou webových aplikácií je ich nezávislosť na konkrétnom zariadení alebo operačnom systéme, pretože ich používanie vyžaduje len webový prehliadač a prístup na internet. Táto vlastnosť zabezpečuje flexibilitu a efektívnosť pri implementácii moderných softvérových riešení pre široké spektrum používateľov [3].

Webové aplikácie a Python

Programovanie webových aplikácií v jazyku Python predstavuje oblasť softvérového vývoja, pri ktorom sa Python používa ako primárny jazyk na tvorbu či už serverovej ale aj klientskej strany, spracovanie požiadaviek a generovanie dynamického obsahu pre používateľov. Python poskytuje robustný ekosystém pre webové technológie vrátane mnohých *frameworkov* a knižníc [4].

Jedným z týchto frameworkov pre Python je Flask, ktorý je takzvaný *mikroframework*, teda minimalistický *framework*, navrhnutý tak, aby poskytoval len základné komponenty potrebné pre webovú aplikáciu a umožňoval rozšírenie pomocou doplnkov podľa potreby. Flask je založený na štandarde WSGI[1], *Web Server Gateway Interface*, ktorý zabezpečuje kompatibilitu s rôznymi webovými servermi a prostrediami [5].

Flask poskytuje jednoduchý spôsob zahájenia vývoja webovej aplikácie a zároveň umožňuje jej následné škálovanie do komplexnejších riešení. Základné komponenty *frameworku* Flask sú Werkzeug(WSGI *toolkit*) a Jinja(šablónovací *engine*), ktoré zabezpečujú spracovanie požiadaviek a generovanie dynamických HTML stránok [5].

Na nasledujúcom výpise 1 je možné vidieť príklad jednoduchej webovej aplikácie naprogramovanej v jazyku Python pomocou *frameworku* Flask. Tento príklad z

[1] štandardizované rozhranie medzi webovým serverom a aplikáciou

oficiálnej dokumentácie *frameworku* Flask demonštruje základné princípy tvorby webovej aplikácie v Pythone [5].

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def hello_world():
    return "<p>Hello, World!</p>"ello World");
}
```

Výpis kódu 1: Ukážka jednoduchej Flask aplikácie

V príklade 1 sa ako prvý krok vykoná import triedy *Flask*, ktorá predstavuje samotnú webovú aplikáciu. Následne sa vytvorí inštancia aplikácie s argumentom *__name__*, ktorý Flask využíva na určenie umiestnenia aplikácie pre načítavanie zdrojov a šablón.

Dekorátor *@app.route("/")* priraďuje funkciu *hello_world()* k hlavnej URL("/"), ktorá po vykonaní požiadavky GET vráti jednoduchý HTML reťazec * Hello, World! *. Tento príklad ilustruje základnú architektúru klient–server, kde server spracuje HTTP požiadavku a vráti odpoveď používateľovi.

1.2.2 Aplikačné programové rozhrania

API je mechanizmus, ktorý umožňuje dvom softvérovým komponentom vzájomne komunikovať prostredníctvom súboru presne definovaných definícií a protokolov. Tým, že API formálne definuje spôsob výmeny informácií medzi aplikáciami, slúži ako kontrakt alebo dohoda o službe, ktorá špecifikuje, ako sa majú formulovať požiadavky a aké odpovede sa očakávajú.

API možno chápať ako abstraktnú vrstvu, ktorá oddeľuje interné detaily implementácie softvéru od externých volaní. V praxi to znamená, že klient(aplikácia, ktorá vysiela požiadavky) a server(aplikácia, ktorá poskytuje odpovede) komunikujú prostredníctvom API podľa predom definovaných pravidiel. Tento model podporuje modulárnosť a umožňuje softvérovým systémom interagovať bez potreby zdieľať svoje vnútorné mechanizmy alebo zdrojový kód.

Architektúra API

API architektúra sa veľmi často interpretuje ako vzťah klient–server, kde klient zasiela požiadavky na server a server následne generuje odpovede. Existuje niekoľko typov API podľa ich návrhu a použitého protokolu:

- SOAP API – využíva protokol SOAP, z angl. *Simple Object Access Protocol* s výmenou správ vo formáte XML. Tieto API boli bežnejšie v minulosti,
- RPC API - *Remote Procedure Call* API, umožňuje volanie funkcií na vzdialenom systéme, teda klient odošle vstup, server vykoná požadovanú funkciu a vráti výsledok.
- WebSocket API – podporuje obojsmernú komunikáciu medzi klientom a serverom s použitím JSON objektov, teda prístup môže byť efektívnejšie pre *real-time*[2] aplikácie,
- REST API – najrozšírenejší typ API, ktorý pracuje na princípoch architektúry *Representational State Transfer*, využíva štandardné HTTP metódy(GET, PUT, DELETE, atď.) a neuchováva stav medzi jednotlivými požiadavkami.

Vyššie uvedené druhy API môžu byť odpoveďou na rôzne potreby vývoja softvéru a navrhujú rôzne prístupy, ako flexibilne a efektívne zabezpečiť komunikáciu medzi systémami.

V tejto práci sa pri nasadzovaní uvažuje najmä REST API, keďže sa jedná o najpopulárnejší a najviac rozšírený druh API.

Celá časť práce pomenovaná ako „Architektúra API“, spolu s úvodnou definíciou podkapitoly, sú spracované podľa [6].

REST API

Architektonický štýl REST API je určený na návrh a implementáciu webových aplikačných rozhraní, pomocou ktorých medzi sebou komunikujú klientské aplikácie a serverové systémy. Ide o všeobecne rozšírený prístup pri vývoji sieťových služieb, ktorý definuje súbor princípov a obmedzení zameraných na dosiahnutie jednoduchej, prehľadnej a škálovateľnej komunikácie medzi jednotlivými softvérovými komponentmi [7].

Základným pojmom pri REST architektúre je takzvaný zdroj, ktorý reprezentuje akýkoľvek dátový alebo informačný objekt, napríklad používateľa, dokument alebo službu. Každý zdroj je jednoznačne identifikovaný prostredníctvom URI, jednotný identifikátor prostriedku z angl. *Uniform Resource Identifier*. Klient pristupuje k týmto zdrojom a manipuluje s nimi pomocou štandardizovaných operácií, pričom REST API funguje na princípe bezstavovej komunikácie. To znamená, že každá požiadavka klienta obsahuje všetky informácie potrebné na jeho spracovanie a server si neuchováva kontext predchádzajúcich interakcií [7].

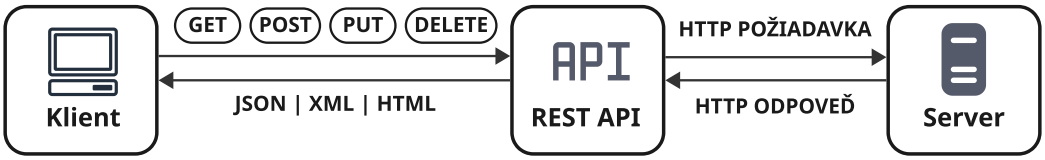
[2]Softvér ktorý reaguje takmer okamžitou odozvou [6]

Aby rozhranie spĺňalo požiadavky REST architektúry, musí dodržiavať viaceré architektonické princípy. Medzi najdôležitejšie patrí oddelenie klienta a servera, jednotné rozhranie, bezstavovosť komunikácie a možnosť ukladania odpovedí do vyrovnávacej pamäte. Dodržiavanie týchto princípov zvyšuje výkonnosť systému a umožňuje jeho jednoduchšiu údržbu a rozširovanie [7].

Komunikácia v REST API prebieha prostredníctvom štandardných HTTP metód, ako sú GET na získanie údajov, POST na vytvorenie nových zdrojov, PUT na ich aktualizáciu a DELETE na ich odstránenie. Stav zdrojov je v odpovediach servera najčastejšie reprezentovaný v dátovom formáte JSON, ktorý je ľahko čitateľný a široko podporovaný [7].

Vďaka svojej jednoduchosti, flexibilite a nezávislosti od konkrétnej platformy sa architektonický štýl REST API stal dominantným riešením pri vývoji moderných webových a *cloud* aplikácií. Tento architektonický štýl umožňuje efektívnu integráciu rôznych systémov a podporuje komunikáciu medzi aplikáciami naprieč rôznymi zariadeniami a prostrediami [7].

Na nasledujúcom obrázku 2 je možné vidieť vizualizáciu REST architektúry.



Obr. 2: Vizualizácia REST API architektúry [7]

API a Python

Na interakciu s aplikačnými programovými rozhraniami je v tejto práci použitý už vyššie uvedený Python. Ak sa jedná o API, Python disponuje širokou škálou použiteľných knižníc ale aj základných funkcionalít ktoré je možné využiť. Práca s API sa delí na dve hlavné časti a to na klienta a server, teda na generovanie a konzumovanie. Na nasledujúcich stranách sú popísané obe časti v Python prostredí.

Konzumovanie, na konzumovanie REST API v jazyku Python sa najčastejšie používa knižnica *requests*, ktorá poskytuje jednoduché rozhranie na odosielanie HTTP požiadaviek a to aj bez potreby detailných znalostí o HTTP protokole [8].

Najbežnejšou HTTP metódou pri konzumácii API je GET, ktorá slúži na načítanie existujúcich zdrojov. Nasledujúci kód 2 ukazuje ako odoslať GET požiadavku na voľne dostupné API nazvané *jsonplaceholder* a spracovať odpoveď [8].

```
import requests
```

```
api_url = "https://jsonplaceholder.typicode.com/todos/1"
response = requests.get(api_url)
print(response.json())
```

Výpis kódu 2: Ukážka GET požiadavky v jazyku Python

Vo vyššie uvedenom príklade sa pomocou metódy *requests.get()* odosiela GET požiadavka na *endpoint*[3], ktorý v tomto prípade vráti konkrétny objekt vo formáte JSON [8].

Po odoslaní HTTP požiadavky server vždy vráti status kód, ktorý klientovi oznamuje výsledok operácie. Klient potom môže podľa tohto kódu rozhodnúť o ďalšom spracovaní výsledku(napr. spracovanie chýb, opakovaný pokus, zobrazenie informácie používateľovi). Pri úspešnej GET požiadavke je typickým status kódom 200, zatiaľ čo pri úspešnom vytvorení zdroja pomocou POST je to 201 [8].

Pre problematiku práce je vhodné uviesť nasledovné, časť konzumovania API spadá do časti automatizačných skriptov, keďže sa väčšinou jedná o Python proces, nie o server, ktorý je potrebné nasadiť.

Generovanie, pri generovaní ide o tvorbu servera, ktorý slúži ako samotné API. Vytváranie takýchto serverov uľahčuje množstvo knižníc v jazyku Python. Táto práca je zameraná najmä na knižnicu FastAPI.

FastAPI je moderný webový framework pre jazyk Python, určený na tvorbu REST API. Je navrhnutý s dôrazom na jednoduchosť, vysoký výkon a bezpečnosť, pričom využíva systém Python typových anotácií pre automatickú validáciu vstupov a generovanie dokumentácie [9].

Medzi kľúčové vlastnosti FastAPI patrí [9]:

- rýchlosť a výkon: vďaka asynchrónnemu spracovaniu je FastAPI jedným z najrýchlejších Python frameworkov pre webové aplikácie,
- automatická dokumentácia: integrované generovanie dokumentácie,
- jednoduchosť použitia: minimalistické rozhranie a deklaratívne definovanie endpointov,
- bezpečnosť: podpora rôznych mechanizmov autentifikácie a autorizácie,
- validácia dát: typové validácie umožňujú automatickú validáciu vstupov a výstupov.

[3] digitálne miesto pre komunikáciu cez API

FastAPI je možné nainštalovať pomocou rôznych správcov Python knižníc ako je napríklad Pip. Na nasledujúcom výpise 3 je možné vidieť inštaláciu knižnice FastAPI, ako aj inštaláciu ASGI, *Asynchronous Server Gateway Interface*[4] servera Uvicorn, ktorý je vlastne webovým serverom, pomocou ktorého je možné spustiť samotný server[9].

```
pip install fastapi
pip install "uvicorn[standard]"
```

Výpis kódu 3: Inštalácia FastAPI a ASGI servera uvicorn

Na nasledujúcom výpise 4 je možné vidieť príklad jednoduchého API vytvoreného pomocou FastAPI [9].

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, FastAPI!"}
```

Výpis kódu 4: Príklad použitia FastAPI

Ukážková aplikácia obsahuje jeden endpoint typu GET, ktorý po dotázaní vráti JSON Objekt s kľúčom *message*, ktorý obsahuje textový reťazec *Hello, FastAPI!*.

1.2.3 Automatizačné skripty

V ponímaní tejto práce sa pod automatizačnými skriptami rozumie každý skript napísaný v programovacom jazyku Python, ktorý je potrebné spúšťať podľa daného harmonogramu. Príkladom môže byť skript, ktorý aktualizuje databázu o nové dáta získané z voľne dostupnej API, pokiaľ sú nové dáta v API dodané každý týždeň. V takomto prípade je potrebné skript púšťať na týždennej báze. V rámci automatizačných skriptov je možné uvažovať aj dve operácie, ktoré na prvý pohľad nemusia evokovať, že sa jedná o automatizačný skript: dátové toky, z angl. *Data Pipelines* a tréning modelov strojového učenia. Práca sa zaoberá najmä dátovými tokmi, keďže tréning modelov strojového učenia sa razantne môže líšiť podľa typu daného modelu a jeho veľkosti.

Dátové toky a Python

Tok dát predstavuje štruktúrovaný sekvenčný proces, ktorým sa dáta získavajú z

[4] štandard pre asynchrónne Python webové aplikácie, ktorý rozširuje WSGI

rôznych zdrojov, transformujú a následne presúvajú do cieľového úložiska, kde sú pripravené na ďalšie využitie, napríklad pre analytické úlohy alebo strojové učenie. Tento proces zabezpečuje, že pôvodné dáta, ktoré často nemajú jednotný formát alebo kvalitu, sú upravené, štandardizované a integrálne pripravené pre kontext podnikových aplikácií alebo výskumných analýz [10].

Tok dát je v informatike chápaný ako metóda automatizovaného spracovania a prenosu dát, ktorá zahŕňa niekoľko hlavných fáz: získavanie dát, transformáciu a uloženie do cieľových systémov, ako sú dátové jazerá alebo dátové sklady. V procese sa často uplatňujú techniky ako filtrovanie, maskovanie, agregácia či čistenie dát s cieľom zabezpečiť ich kvalitu, konzistenciu a použiteľnosť pre následné analytické úlohy [10].

Nasledujúci kód 5 predstavuje zjednodušenú implementáciu dátového toku v programovacom jazyku Python s využitím knižnice *pandas*, ktorá je bežne používaná na spracovanie a analýzu štruktúrovaných dát. Cieľom implementácie je demonštrovať základné fázy toku dát, konkrétne získavanie dát, transformáciu a agregáciu údajov.

```
import pandas as pd

# Získanie dát
data = pd.DataFrame({
    "sensor_id": ["S1", "S2"],
    "timestamp": ["2026-01-10 10:00:00", "2026-01-10 10:00:00"],
    "temperature": [21.5, -99.0]
})

# Transformácia dát
cleaned = data[(data["temperature"] > -30) & (data["temperature"] < 60)]
cleaned["timestamp"] = pd.to_datetime(cleaned["timestamp"])

# Agregácia
result = cleaned.groupby("sensor_id")["temperature"].mean().reset_index()
result.rename(columns={"temperature": "avg_temperature"}, inplace=True)

# Uloženie
print(result)
```

Výpis kódu 5: Príklad dátového toku

V prvej fáze dochádza k získaniu dát prostredníctvom vytvorenia dátovej tabuľky simulujúcej merania zo senzorov, ktoré môžu obsahovať aj neplatné hodnoty. Následne sú dáta transformované odstránením chybných meraní a konverziou časového údaju do jednotného dátového formátu. V záverečnej fáze dátového toku sú dáta

agregované podľa identifikátora senzora, pričom sa vypočíta priemerná hodnota teploty, čím vzniká výstup pripravený na ďalšie analytické spracovanie.

Existujú aj rôzne nástroje alebo *frameworky* pre prácu s dátovými tokmi v jazyku Python, jedným z nich a veľmi rozšíreným je Apache Airflow. Jedná sa o platformu, ktorá dovoľuje programaticky vytvárať, plánovať a monitorovať komplexné dátové toky.

Tréning modelov strojového učenia a Python

Tréning modelu strojového učenia predstavuje proces učenia modelu strojového učenia, v ktorom sa model optimalizuje tak, aby dosahoval požadované výsledky pri predikcii alebo rozhodovaní na základe vstupných dát relevantných pre jeho budúce úlohy. Tréning je považovaný za kritickú fázu životného cyklu modelov strojového učenia, pretože práve v ňom dochádza k úprave vnútorných parametrov modelu prostredníctvom algoritmických postupov tak, aby sa minimalizovala chyba pri predikcii výsledkov [11].

V kontexte strojového učenia, učenie znamená úpravu parametrov modelu — najmä váh a posunov v matematických funkciách predstavujúcich model — na základe trénovacích dát, aby model generoval presné výstupy. Cieľom tejto úpravy je minimalizovať funkciu straty, ktorá kvantifikuje odchýlku medzi predikciami modelu a skutočnými cieľovými hodnotami. Keď hodnota funkcie straty klesne pod prednastavenú hranicu, model sa považuje za dostatočne „natrénovaný“ [11].

Proces tréningu zahŕňa cyklické kroky, ktoré opakovane spracúvajú dáta, merajú chybu, optimalizujú parametre modelu a overujú výkon pomocou validačných datasetov. Pomocou tejto iterácie sa model postupne zlepšuje až do dosiahnutia uspokojivej presnosti alebo stability učenia [11].

1.3 Použitá sada nástrojov pre nasadzovanie aplikácií

Pri nasadzovaní Python aplikácií do produkcie je možné voliť z pomedzi množstva technológií a nástrojov, ktoré túto úlohu zjednodušujú, respektíve štandardizujú. Od voľby správcu prostredia pre Python, teda nástroj pomocou ktorého je možné obstarávať knižnice, distribuovať aplikáciu alebo aj voliť verziu jazyka Python až po výber technológie vďaka ktorej je možné aplikáciou takzvane kontajenerizovať a následne nasadiť na kláster ktorý je orchestrovaný ďalšiou aplikáciou.

V následujúcich podkapitolách sú všetky vyššie uvedené termíny vysvetlené a taktiež sa v nich nachádza popis vybranej technológie pre problematiku práce.

1.3.1 Python správa prostredia

Správa prostredia je v kontexte vývoja softvéru kľúčovým aspektom zabezpečenia konzistencie, reprodukovateľnosti a stability aplikácií. V prostredí Python, ktoré je dynamické a flexibilné, sa správa prostredí stáva nevyhnutnou pre efektívny vývoj, testovanie a nasadzovanie aplikácií. Nesprávne nakonfigurované prostredie môže viesť k problémom so závislosťami, nekompatibilitou knižníc a nepredvídateľným chybám pri prevádzke aplikácií [12].

Python umožňuje vytváranie izolovaných **virtuálnych prostredí**, ktoré slúžia na oddelenie závislostí jednotlivých projektov. Virtuálne prostredia zabezpečujú, že konkrétna aplikácia používa len tie knižnice a verzie, ktoré sú pre ňu relevantné, čím sa minimalizuje riziko konfliktov medzi projektmi. V Python ekosystéme sú najpoužívanejšie nástroje na správu virtuálnych prostredí *Venv* a *Virtualenv* [12].

venv je štandardný modul Pythonu od verzie 3.3, ktorý umožňuje rýchle a jednoduché vytváranie izolovaných prostredí. Každé prostredie obsahuje samostatnú inštaláciu interpretera[5] Python a vlastnú štruktúru adresárov pre knižnice [12].

Nástroje na správu prostredia

Pip, je štandardný a historicky najrozšírenejší správca prostredia pre jazyk Python. Je súčasťou oficiálnej distribúcie Python a predstavuje základný nástroj na inštaláciu balíkov z repozitára PyPI, *Python Package Index*. Pip umožňuje inštaláciu, aktualizáciu a odstraňovanie Python balíkov, pričom primárne pracuje so súborom *requirements.txt*, ktorý definuje závislosti projektu, nepokrýva správu virtuálnych prostredí ani uzamykanie verzií [13].

Uv, je moderný správca balíkov a prostredí pre Python, vyvíjaný spoločnosťou Astral. Je implementovaný v jazyku Rust a navrhnutý ako výkonnostne optimalizovaná náhrada za pip a súvisiace nástroje. kombinuje funkcionalitu pip a manažéra virtuálnych prostredí, poskytuje extrémne rýchle riešenie závislostí, podporuje uzamykanie verzií pre deterministické vytváranie prostredí a je plne kompatibilný s PyPI [14].

Poetry, je správca prostredí a projektov pre Python, ktorý kladie dôraz na reprodukovateľnosť, správu verzií a konzistenciu vývoja. využíva súbory *pyproject.toml* a *poetry.lock*, poskytuje pokročilé riešenie závislostí, automaticky spravuje virtuálne prostredia a podporuje publikovanie balíkov [15].

[5] program, ktorý číta a priamo vykonáva kód napísaný v jazyku Python

Z vyššie uvedených nástrojov na správu prostredia, je pre účely nasadzovania pomocou veľkého jazykového modelu najviac vhodný pip, kvôli jeho rozšírenosti a jednoduchosti.

1.3.2 Kontajner a kontajnerizácia aplikácií

Kontajnerizácia predstavuje prístup k vývoju a nasadzovaniu softvéru, pri ktorom sú aplikácie spolu so svojimi závislosťami prevedené do izolovaných prostredí nazývaných kontajnery. Tieto prostredia umožňujú spúšťať aplikácie konzistentne naprieč rôznymi výpočtovými platformami bez potreby úprav konfigurácie alebo závislostí [16].

Na rozdiel od tradičnej virtualizácie, kde je každá aplikácia spustená vo vlastnom virtuálnom stroji s kompletným operačným systémom, kontajnery zdieľajú jadro hostiteľského operačného systému. Vďaka tomu sú výrazne menšie, rýchlejšie sa spúšťajú a efektívnejšie využívajú systémové zdroje [17]. Tento model znižuje režijné náklady a umožňuje vysokú frekvenciu nasadenia aplikácií na jednom fyzickom alebo virtuálnom serveri.

Kontajnerizácia sa stala základným stavebným prvkom moderných aplikácií a mikroservisných architektúr. Umožňuje rozdelenie monolitických aplikácií na menšie, nezávisle nasadzovateľné služby, čím sa zvyšuje flexibilita, škálovateľnosť a odolnosť systému voči zlyhaniam [18].

Kontajner

Kontajner je izolovaná spustiteľná jednotka, ktorá obsahuje aplikáciu, jej knižnice, prostredie a konfiguračné súbory potrebné na správne fungovanie. Izolácia je dosiahnutá prostredníctvom mechanizmov operačného systému, ako sú menný priestor a kontrolné skupiny, ktoré zabezpečujú oddelenie procesov, siete a využívania zdrojov [19].

Z technického hľadiska je kontajner spustenou inštanciou kontajnerového obrazu. Kontajnerový obraz predstavuje nemennú šablónu, ktorá definuje presné zloženie aplikačného prostredia. Tento prístup umožňuje verziovanie, opakovateľnosť a jednoduchú distribúciu aplikácií medzi rôznymi prostrediami [17].

Medzi hlavné výhody kontajnerov patrí rýchly štart, nízke pamäťové nároky a vysoká prenositeľnosť. V porovnaní s virtuálnymi strojmi poskytujú kontajnery lepší pomer medzi výkonom a spotrebou zdrojov, čo je kľúčové najmä v prostrediach s vysokými nárokmi na škálovanie a dostupnosť [20].

Docker ako platforma pre kontajnerizáciu aplikácií

Docker je platforma s otvoreným zdrojovým kódom určená na automatizáciu tvorby,

nasadzovania a správy kontajnerizovaných aplikácií. Umožňuje vývojárom transformovať aplikáciu spolu so všetkými jej závislosťami na Docker obrazu, ktorý je následne možné spustiť ako kontajner v ľubovoľnom kompatibilnom prostredí [17].

Docker architektúra pozostáva z niekoľkých základných komponentov. Docker Engine predstavuje prostredie zodpovedné za vytváranie a spúšťanie kontajnerov. Docker obrazy slúžia ako nemenné šablóny aplikácií a Docker kontajnery sú ich konkrétne bežiace inštancie [16]. Tento model umožňuje jednoduchú reprodukovateľnosť aplikačného prostredia a znižuje riziko chýb spôsobených rozdielmi medzi vývojovým a produkčným prostredím.

Docker je použitý ako kontajnerizačná aplikácia pri riešení problematiky tejto práce.

1.3.3 Orchestrácia kontajnerov

S rastúcim využívaním kontajnerizovaných aplikácií v moderných softvérových systémoch vznikla potreba ich efektívnej správy vo veľkom rozsahu. Zatiaľ čo kontajnerové prostredia, ako napríklad Docker, zabezpečujú vytváranie a spúšťanie jednotlivých kontajnerov, neposkytujú dostatočné mechanizmy na koordináciu tisícov kontajnerov v distribuovanom prostredí. Túto úlohu plní orchestrácia kontajnerov, ktorá umožňuje automatizované riadenie životného cyklu kontajnerizovaných aplikácií v rámci klastrov a ich výpočetových uzlov [21].

Orchestrácia kontajnerov zahŕňa kľúčové funkcie, medzi ktoré patrí plánovanie kontajnerov na dostupné uzly, automatické zotavenie po zlyhaní, vyvažovanie záťaže, škálovanie aplikácií a správa sieťovej komunikácie. Tieto mechanizmy sú nevyhnutné pre prevádzku aplikácií a mikroservisných architektúr, ktoré kladú vysoké nároky na dostupnosť, flexibilitu a efektívne využívanie zdrojov [22].

Kubernetes ako platforma pre orchestráciu kontajnerov

Kubernetes je platforma s otvoreným zdrojovým kódom určená na automatizáciu nasadzovania, škálovania a správy kontajnerizovaných aplikácií. Bol pôvodne vyvinutý spoločnosťou Google na základe interného systému Borg a v roku 2014 bol vydaný ako projekt s otvoreným zdrojovým kódom. V súčasnosti je Kubernetes spravovaný organizáciou CNCF, *Cloud Native Computing Foundation* a považuje sa za štandard v oblasti orchestrácie kontajnerov [21].

Základným princípom Kubernetes je deklaratívny model správy infraštruktúry. Používateľ definuje požadovaný stav systému prostredníctvom konfiguračných súborov vo formáte YAML alebo JSON, zatiaľ čo samotná platforma zabezpečuje, aby bol tento stav kontinuálne udržiavaný. Kubernetes automaticky reaguje na zmeny v

prostredí, ako sú zlyhania uzlov alebo zvýšené zaťaženie aplikácií, čím zabezpečuje vysokú dostupnosť a odolnosť systému [21].

Kubernetes zároveň poskytuje jednotnú abstrakčnú vrstvu nad infraštruktúrou, čo umožňuje konzistentnú prevádzku aplikácií naprieč rôznymi *cloudovými* a *on-premise*[6] prostrediami [21].

Kubernetes je použitý ako orchestrátor kontajnerov pri problematike tejto práce.

1.3.4 Proces nasadenia kontajnerizovanej aplikácie

Napriek tomu, že kontajnerizácia významne zjednodušuje distribúciu a reprodukovateľnosť aplikácií, samotný proces nasadenia pozostáva z viacerých krokov, ktoré vyžadujú explicitné zásahy zo strany operačného inžiniera. Kontajnerizácia totiž nepredstavuje automatické nasadenie aplikácie, ale poskytuje štandardizovaný artefakt, s ktorým je možné ďalej pracovať v procese nasadzovania.

Prvým krokom je vytvorenie konfiguračného súboru Dockerfile, ktorý definuje prostredie aplikácie, jej závislosti, spôsob zostavenia a spôsob spustenia. Tento krok predstavuje manuálny zásah, pretože vývojár musí explicitne určiť všetky komponenty potrebné pre správne fungovanie aplikácie, vrátane výberu základného obrazu, inštalácie knižníc, kopírovania zdrojového kódu a definovania vstupného bodu aplikácie.

Nasleduje proces zostavenia kontajnerového obrazu, počas ktorého je Dockerfile transformovaný do nemenného binárneho artefaktu – Docker obraz. Tento krok je potrebné iniciovať manuálne, napríklad prostredníctvom príkazu *docker build* alebo pomocou automatizovaného nástroja v rámci CI/CD pipeline. Výsledkom je verzionovaný obraz, ktorý predstavuje distribuovateľnú jednotku aplikácie.

Ďalším krokom je uloženie vytvoreného obrazu do kontajnerového registra, ako je napríklad Docker Hub alebo súkromný register. Tento proces zahŕňa autentifikáciu do registra, označenie obrazu vhodným identifikátorom(tag) a jeho odoslanie pomocou príkazu *docker push*. Tento krok je nevyhnutný na to, aby bol obraz dostupný pre cieľové prostredie, napríklad pre produkčný server alebo orchestrátor kontajnerov.

Nasadenie aplikácie následne vyžaduje definovanie spôsobu spustenia kontajnera v cieľovom prostredí. To zahŕňa konfiguráciu parametrov, ako sú sieťové nastavenia, mapovanie portov, premenné prostredia, prístup k perzistentným dátam a definovanie požiadaviek na systémové zdroje. V prípade použitia orchestrátora, ako je napríklad Kubernetes, je potrebné vytvoriť konfiguračné manifesty, ktoré určujú požadovaný

[6] IT infraštruktúra, kde sú servery, softvér a dáta fyzicky umiestnené a prevádzkované priamo danou spoločnosťou

stav aplikácie, vrátane počtu replík, spôsobu aktualizácie a pravidiel obnovy v prípade zlyhania.

1.4 Generatívna umelá inteligencia

Celá myšlienka návrhu nového prístupu pri nasadzovaní aplikácií je postavená na podmnožine generatívnej umelej inteligencie. Jedná sa o podmnožinu veľkých jazykových modelov, ktoré v posledných rokoch získavajú stále viac popularity a nachádzajú nové využitia.

Generatívna umelá inteligencia predstavuje zoskupenie výpočtových techník schopných generovať zdanlivo nový a zmysluplný obsah, ako je text, obraz, zvuk alebo programový kód, na základe tréningových dát [23]. Na rozdiel od tradičných analytických foriem umelej inteligencie, ktoré sa zameriavajú najmä na klasifikáciu alebo predikciu, generatívna umelá inteligencia produkuje nové výstupy, ktoré môžu byť nerozoznateľné od ľudskej tvorby.

Na **modelovej úrovni** sa nachádzajú samotné generatívne modely, ako sú LLM, vVeľké jazykové Mmodely, z angl. *Large Language Models*, difúzne modely alebo generatívne adversariálne siete. Medzi významné architektúry patria transformátory [24] a variacionálne autoenkodéry [25, 26].

Na **systémovej úrovni** sa generatívne modely integrujú do komplexných softvérových systémov, ktoré zahŕňajú používateľské rozhrania, dátové toky a infraštruktúru. Príkladom sú konverzačné systémy ako ChatGPT, AI Agenti alebo vývojárske nástroje ako GitHub Copilot [27]. Táto úroveň je rozhodujúca pre použiteľnosť, škálovateľnosť a adopciu generatívnej AI v praxi.

Aplikačná úroveň reprezentuje konkrétne prípady využitia generatívnej AI v organizáciách, napríklad pri generovaní marketingového obsahu, softvérového kódu alebo vzdelávacích materiálov [23]. Na tejto úrovni sa generatívna AI stáva súčasťou socio-technických systémov, kde podporuje alebo rozširuje ľudské schopnosti.

1.4.1 AI Agent

Nasledujúca kapitola sa zaoberá vybranými architektúrami a prístupmi, ktoré boli zvolené na overenie konceptu a môžu byť možným riešením definovaného problému.

Na rozdiel od tradičných prístupov, pri ktorých sú akcie a kroky programu pevne určené vopred definovanou logikou, pri agentoch tieto kroky ovplyvňuje alebo úplne určuje veľký jazykový model. Za agenta teda možno považovať aj relatívne jednoduchý systém, pokiaľ veľký jazykový model rozhoduje aj napríklad o

postupnosti vopred definovaných akcií. Rozhodujúcim kritériom je skutočnosť, či má LLM priamy vplyv na rozhodovanie v aplikácii [28].

S pojmom agent úzko súvisí aj koncept agentickosti, ktorý označuje mieru autonómie jazykového modelu pri riadení systému. Teda na nižších úrovniach agentickosti LLM vykonáva obmedzené rozhodovanie v rámci pevne preddefinovaných akcií, a naopak na vyšších úrovniach agentickosti LLM rozhoduje o opakovaní akcií, spracúva spätnú väzbu, pracuje s podmienkami a samostatne určuje ukončenie činnosti aplikácie [28].

Agent je teda chápaný nie ako konkrétny algoritmus, ale ako architektonický princíp, v ktorom LLM plní úlohu centrálneho rozhodovacieho mechanizmu.

1.4.2 LLM nástroje

Ako bolo definované v predchádzajúcej časti, na základe výstupu veľkého jazykového modelu, je možné rozhodovať o nasledujúcom kroku aplikácie. Avšak pomocou LLM je možné aj vykonávať rozsiahlejšie úkony na vyriešenie danej úlohy ako napríklad celé funkcie a je možné nechať rozhodnutie o tom, ktorú funkciu vykonať a kedy, na veľký jazykový model. Takéto funkcie ktoré môže LLM spustiť alebo presnejšie vyžiadať si ich spustenie a následne obdržať výsledok sa nazývajú nástroje, z angl. *tools* [29].

Podľa definície je LLM nástroj, pomenovaná funkcia, ktorá je registrovaná a dostupná na zavolanie pre daný model. Pod pojmom registrovaná sa rozumie obsadenie popisu funkcie ktorý má štrukturovaný formát v kontexte promptu modelu [29].

Napríklad nižšie uvedená štruktúra 6 je vložená do promptu modelu a vďaka tejto informácii má model možnosť, vyžiadať spustenie daného nástroja na základe vlastného rozhodnutia.

```
{
  "name": "getWeather",
  "description": "Vráti počasie pre dané mesto",
  "parameters": {
    "type": "object",
    "properties": {
      "city": { "type": "string" }
    },
    "required": ["city"]
  }
}
```

Výpis kódu 6: Príklad definície LLM nástroja

Registráciu LLM nástrojov je možné popísať ako pridanie klauzule do promptu ktorá dovoľuje modelu vrátiť názov jednej z definovaných funkcií spolu so vstupnými argumentami. Po vrátení jedného z nástrojov modelom, je už na samotnom systéme, respektíve používateľovi vykonať túto funkciou a spustiť interakciu s modelom znova ale teraz už aj s odpoveďovou, ktorú vrátil daný nástroj.

Pokiaľ bol veľký jazykový model pred-trénovaný na používanie takýchto nástrojov tak sa jedná o model s takzvaným *tool callingom*. Pri takomto modeli nie je potrebné zahŕňať klauzulu definujúcu možnosť využitia nástrojov do promptu ale stačí iba pokynúť definíciu nástrojov v danom formáte.

1.4.3 Multi-agent Systém

Multi-agent systém predstavuje architektúru, v ktorej viacero agentov spolupracuje na riešení danej úlohy. Základnou myšlienkou tohto prístupu je rozdelenie zodpovednosti medzi špecializovanými agentmi, z ktorých každý je navrhnutý na riešenie daného typu problému. Takéto rozdelenie umožňuje zvýšiť modularitu systému, zlepšiť jeho škálovateľnosť a zároveň dosiahnuť väčšiu efektivitu v porovnaní s monolitickým modelov v niektorých prípadoch. Je však ale vhodné podotknúť že nie každá komplexná úloha si vyžaduje Multi-agent architektúru [30].

Táto architektúra je vo väčšine prípadov zvolené z nasledujúcich dôvodov:

- manažment kontextu: možnosť špecializovania kontextu pre daného agenta, vďaka čomu sa veľkosť kontextu ktorá je obmedzené znižuje, tým pádom aj latencia,
- distribuovaný vývoj: umožňuje nezávislý vývoj častí systému,
- paralelizácia: možnosť konkuretného prepoužitia špecializovaných agentov,
- veľké množstvo nástrojov: pokiaľ má samostatný agent primnoho nástrojov a nezvláda ich korektne používať.

Existuje mnoho, vzorov, respektíve architektúr definujúcich Multi-agent systémy a každá z nich je vhodná na iné použitie. Podľa [30], sú rozlišované následné druhy architektúr:

1. vnorení agenti,
2. machanizmus odovzdávania,
3. mechanizmus dynamických zručnostií,
4. smerovač.

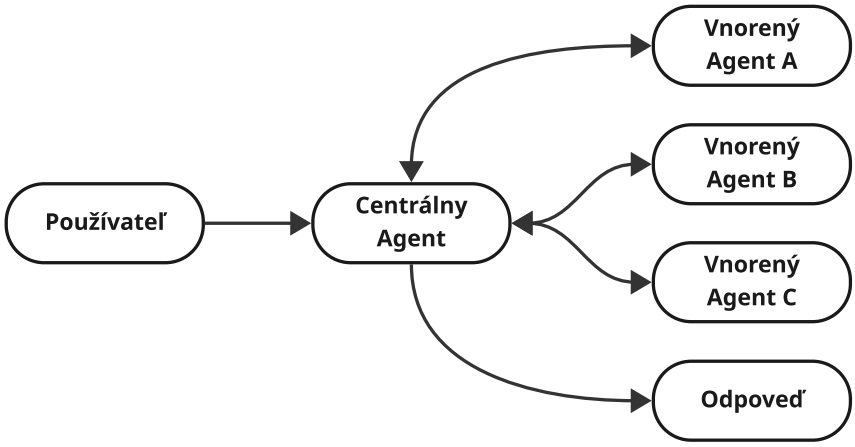
1.4.3.1 Vnorení agenti

Pri architektúre využívajúcej vnorených agentov sa používa tak zvaný centrálny agent, ktorý koordinuje vnorených agentov tým že ich využíva ako LLM nástroje. Centrálny agent rozhoduje o tom, ktorý vnorený agent bude zavolaný, aký vstup bude použitý a ako následne budú výsledky skombinované. Vnorení agenti sú bez-stavový, teda nemajú vedomosť o predchádzajúcich interakciách, keďže všetká konverzačná pamäť je spravovaná centrálnym agentom. Vďaka tomu je využívaná takzvaná izolácia kontextu, kedy každý vnorený agent pri zavolaní pracuje s čistým kontextom [30].

Kľúčovými charakteristikami architektúry vnorených agentov sú:

- centralizovaná kontrola: celá premávka je smerovaná centrálnym agentom,
- nepriame zásahy používateľa: vnorení agenti vracajú výsledok centrálnemu agentovi, nie používateľovi
- agenti sú volaní ako nástroje,
- paralelné spúšťanie vnorených agentov: centrálny agent môže spúšťať viacero vnorených agentov súčasne.

Na nasledujúcom obrázku 3 je možné vidieť vizualizáciu architektúry využívajúcej vnorených agentov.



Obr. 3: Vizualizácia Multi-agent architektúry využívajúcej vnorených agentov [30]

1.4.3.2 Mechanizmus odovzdávania

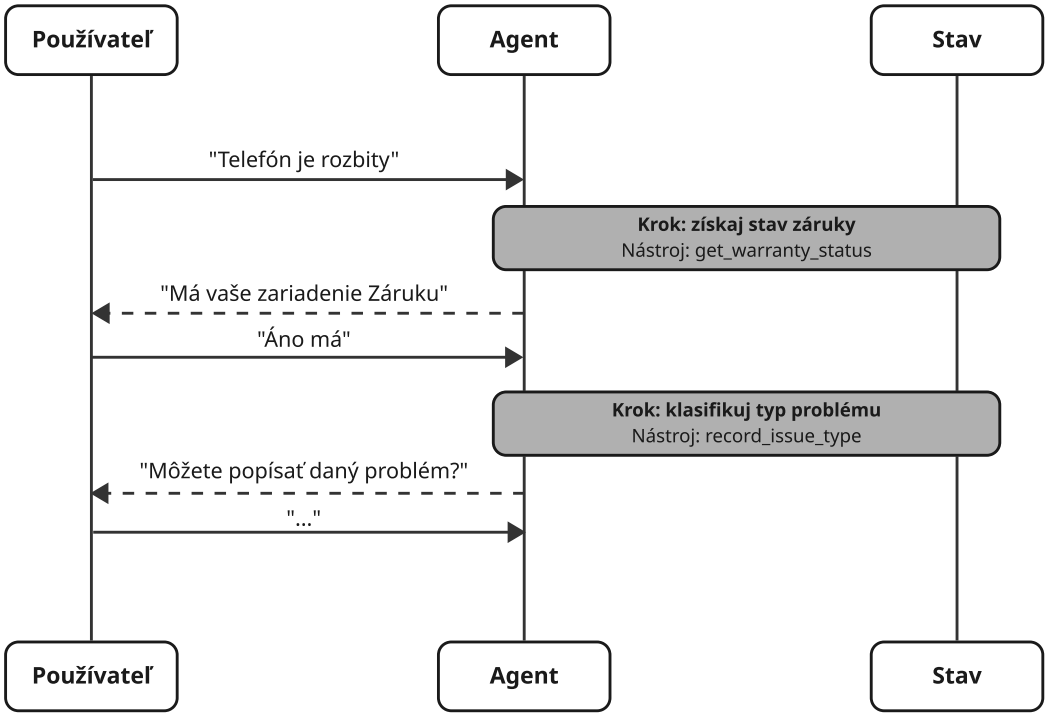
Pri architektúre využívajúcej mechanizmus odovzdávania sa správanie systému dynamicky mení v závislosti od stavu. Základným mechanizmom je aktualizácia

stavovej premennej nástrojmi, ktorá pretrváva medzi jednotlivými interakčnými krokmi. Systém túto premennú následne vyhodnocuje a na jej základe upravuje správanie, buď aplikuje odlišnú konfiguráciu(systémový prompt, dostupné nástroje) alebo zemnu využívaného agenta. Tento mechanizmus umožňuje jednak odovzdávanie riadenia medzi samostatnými agentmi, ako aj dynamické zmeny konfigurácie v rámci jedného agenta [30].

Kľúčovými charakteristikami mechanizmu odovzdávania sú:

- správanie riadené stavom,
- prechody medzi stavmi závislé na nástrojoch,
- priama interakcia s používateľom,
- perzistetný stav konverázcie.

Na nasledujúcom obrázku 4 je možné vidieť vizualizáciu mechanizmu odovzdávania.



Obr. 4: Vizualizácia Multi-agent architektúry využívajúcej mechanizmus odovzdávania [30]

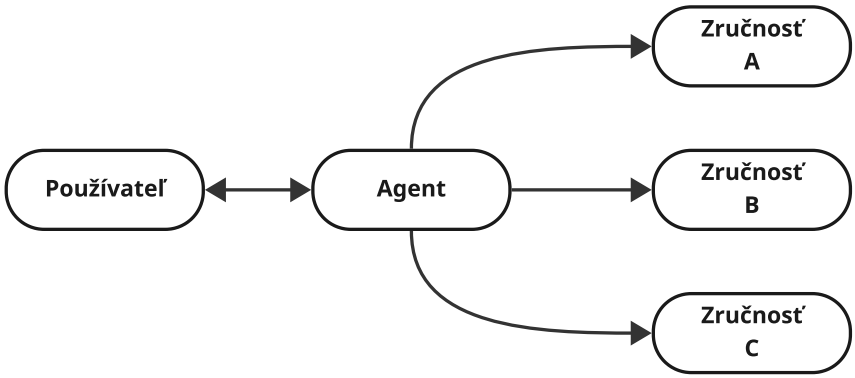
1.4.3.3 Mechanizmus dynamických zručností

Architektúra využívajúca mechanizmus dynamických zručnosti, poskytuje možnosť agentovi, využiť takzvanú zručnosť a rozšíriť teda dané schopnosti podľa potreby. Zručnosťou sa väčšinou rozumie, promptom riadená špecializácia ktorú môže agent využiť [30].

Kľúčovými charakteristikami machanizmu dynamických zručností sú:

- promptami riadené špecializácia,
- zručnosti sú dostupné na základe kontextu alebo používateľovej potreby,
- rôzne tými, môžu spravovať rôzne vlastnosti nezávisle,
- mechanizmus dynamických zručnosti je jednoduchší ako vnorení agenti.

Na nasledujúcom obrázku 5 je možné vidieť vizualizáciu dynamických zručností.



Obr. 5: Vizualizácia Multi-agent architektúry využívajúcej mechanizmus dynamických zručností

[Vizualizácia Multi-agent architektúry využívajúcej mechanizmus dynamických zručností [30]]

1.4.3.4 Smerovač

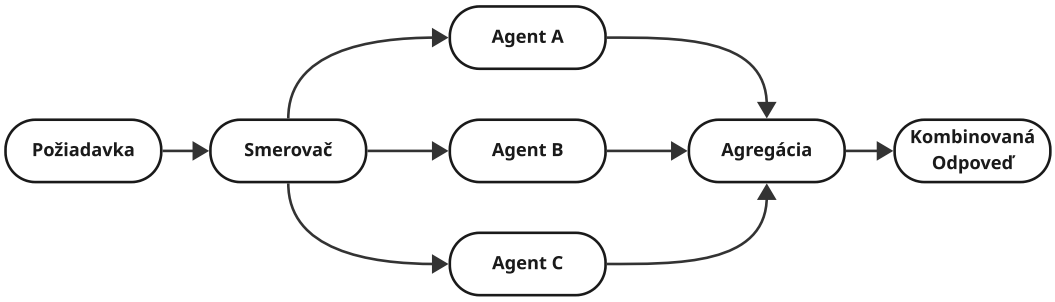
Pri architektúre využívajúcej smerovač, sa pred samotnými agentami nachádza tak zvaný smerovací krok, ktorý spracuje používateľovu požiadavku a na základe nej spustí jedného alebo niekoľko agentov zameraných na špecifickú doménu paralelne [30].

Kľúčovými charakteristikami architektúry využívajúcej smerovač sú:

- smerovač analyzuje požiadavku od používateľa,

- žiadny alebo viacero agentov môže byť spustených súčastne,
- výsledky z jednotlivých agentov sú agregované do jednej odpovede.

Na nasledujúcom obrázku 6 je možné vidieť vizualizáciu architektúry využívajúcej smerovač.



Obr. 6: Vizualizácia Multi-agent architektúry využívajúcej smerovač [30]

Rozhodujúcim faktorom pri návrhu riešenia, je podmienka, maximálne jednej interakcie s používateľom, respektíve poskytnutie vstupného zdrojového kódu používateľom. Tento faktor vylučuje veľké množstvo možností a prístupov na experimentovanie.

Pri navrhovaní riešenia, ktorého cieľom je vytvoriť Docker obraz na základe zdrojového kódu je potrebné plniť takmer rovnaké kroky aké by realizoval operačný inžinier. Týmito krokmi sú, analýza zdrojového kódu, vytvorenie Dockerfile, vytvorenie Docker obrazu, otestovanie Docker obrazu a nahranie na vzdialený repozitár z ktorého môže byť daný obraz využitý.

Niektoré z týchto krokov môžu byť ponímané ako samostatné nástroje, a to napríklad, načítanie zdrojového priečniku a jeho celého obsahu, vytvorenie Dockerfile pomocou príkazu *docker build*, spustenie Docker kontajnera a nahranie na vzdialený repozitár.

2 Súčasný stav

Existuje veľké množstvo dostupných riešení, ktoré sa zaoberajú automatizáciou nasadzovania do produkcie s minimálnym zásahom používateľa. Cieľom takéhoto nasadenia pomocou služby je odovzdanie zdrojového kódu na vstupe a na výstupe obdržanie zdroja, pomocou ktorého je možné monitorovať alebo využívať aplikáciu. Pri tomto procese sa môže jednať o webovú aplikáciu ako aj napríklad o automatizačný skript, ktorý je spúšťaný na pravidelnej báze. Pre zadefinovanie súčastného stavu na účely tejto práce sú najviac relevantné dve skupiny služieb, PaaS, platforma ako služba, z angl. *Platform as a Service* a takzvané služby využívajúce LLM na vygenerovanie zdrojov potrebných na nasadenie.

2.1 PaaS

Platforma ako služba je model poskytovania výpočtových služieb, ktorý umožňuje vývoj, testovanie, nasadzovanie a prevádzku aplikácií prostredníctvom poskytovanej platformy, pričom používateľ nie je zodpovedný za správu technického prostredia akým je hardvér a potrebný infraštruktúrny softvér, na ktorý sú aplikácie nasadené. PaaS umožňuje vývojárom sústrediť sa výlučne na tvorbu aplikácií, skracuje čas ich nasadenia a znižuje náklady spojené s ich prevádzkou [31].

Aplikácie vytvorené na PaaS platforme môžu byť nasadené priamo do produkcie bez potreby manuálnych zásahov do technického prostredia, čo znižuje riziko chýb a nekonzistencií medzi vývojovým a produkčným prostredím. Tento prístup prispieva k rýchlejšiemu uvedeniu aplikácií do prevádzky a podporuje častejšie a spoľahlivejšie aktualizácie aplikácií [31].

Medzi PaaS platformy, ktoré ponúkajú žiadanú funkcionalitu pre Python, patria najmä:

- **Render**: https://render.com,
- **Railway**: https://railway.com,
- **Fly.io**: https://fly.io/,
- **Google Cloud Run**: .https://cloud.google.com/run?hl=sk

V nasledujúcej tabuľke 1 a tabuľke 2 je možné vidieť porovnanie vyššie uvedených platforiem. Pri porovnaní bol kladený dôraz na vlastnosti, ako sú podpora jazyka Python, podporované druhy Python aplikácií, podpora automatizačných skriptov, automatické nasadenie projektu na základe zdrojového kódu a cena.

Tabuľka 1: Porovnanie PaaS s funkciou nasadenia zo zdrojového kódu [32–35]

Služba	Python	Aplikácie	Časovanie	Auto nasadenie
Render	áno	web, as	áno	nie
Railway	áno	web, as	áno	nie
Fly.io	áno	web	nie	nie
Google Cloud Run	áno	web, as	áno	nie

Definícia jednotlivých stĺpcov v tabuľke 1 je nasledovná. Stĺpec *Python* obsahuje hodnotu áno, pokiaľ daná platforma podporuje nasadzovanie aplikácií naprogramovaných pomocou jazyka Python. Posledný stĺpec *Auto Nasadenie* rozhoduje o tom, či daná platforma dokáže automaticky určiť, o aký typ aplikácie sa jedná a na základe toho vykonať nasadenie. Keďže ani jedna z platforiem túto funkcionalitu neumožňuje, je potrebné uvažovať aj stĺpec *Aplikácie*, ktorý hovorí o tom, aký druh aplikácie je možné nasadiť pomocou danej platformy, skratka *as* znamená v tomto prípade automatizačný skript. Stĺpec *Časovanie*, udáva či je v platforme možné zadefinovať vykonanie úlohy v daný čas.

Pri vyššie uvedených službách je potrebné podotknúť, že nie všetky sú schopné nasadiť daný druh aplikácie naprieč všetkými frameworkami a knižnicami, napríklad Fly.io dokáže nasadiť výhradne aplikácie typu FastAPI, Flask a Streamlit. V tabuľke sa táto informácia však neuvádza, keďže tieto zoznamy sú dohladateľné na stránkach každej platformy.

Tabuľka 2: Porovnanie ceny PaaS schopných nasadenia zo zdrojového kódu [32–35]

Služba	Fixná Zložka(€)	Výpočtový výkon(€)	Spolu Mesiac(€)
Render	16	193	209
Railway	20	206	226
Fly.io	37	36	73
Google Cloud Run	0	45	45

Ako je možné vidieť v predchádzajúcej tabuľke 2, pri PaaS službách sa zvyčajne platí fixná mesačná zložka, na základe ktorej sa sprístupňujú rôzne funkcionality ako je napríklad, počet projektov a používateľov pre danú platformu. V zobrazenom porovnaní sú uvažované iba fixné zložky, ktoré dovoľujú viac ako 5 projektov a viac ako 5 používateľov, teda cieľom bolo simulovať reálny tím a vývoj. Pri cenách za zdroje je metodika komplikovanejšia, keďže existuje rozsiahla škála možných zdrojov

ako jadrá procesora, operačná pamät, úložný priestor, grafický výkon a podobne. V prieskume bola cena vypočítaná ako súčet ceny za 4 jadrá procesora a 8GB operačnej pamäte ak by boli využívané 30 dní neustále. Tieto dva typy zdrojov boli vybrané z dôvodu, že práca sa venuje najmä webovým aplikáciam a automatizačným skriptom ktoré nie sú tak náročné na zdroje a nepotrebujú grafický výpočtový výkon ako napríklad trénovanie modelov strojového učenia. Je dôležité podotknúť že sumy boli zaokrúhľované nahor na celé čísla a sú uvedené bez dane z pridanej hodnoty.

Na základe vykonaného prieskumu je možné konštatovať, že žiadna z dostupných služieb nepodporuje automatické nasadenie, teda prípad, kedy používateľ iba poskytne ľubovolný Python kód a ten je následne nasadený. Používateľ si musí voliť z preddefinovaných šablón alebo do-definovať množstvo vecí, ako vstupný bod, dodatočné knižnice(ak túto funkcionalitu daná služba podporuje), typ aplikácie a podobne. Ale samotné nasadenie Python aplikácie ako také či už webovej aplikácie alebo automatizačného skriptu je pomerne bežná záležitosť.

Avšak z druhej tabuľky vyplýva, že tento proces môže byť pomerne finančne náročný, keďže cena zdrojov sa udáva za jeden projekt aj keď fixná zložka môže pokryť viacero projektov. Taktiež pri väčších projektoch sú väčšinou nutné dodatočné poplatky. Z objektívneho hľadiska by vytvorenie takejto platformy predstavovalo mimoriadne finančne náročný proces, najmä vzhľadom na vysoké technické a prevádzkové požiadavky, ktoré by si vyžadovali značné investície do infraštruktúry, softvérových riešení a odborného personálu. Tento problém je mimoriadne náročný, pričom každý projekt sa môže vzhľadom na špecifické požiadavky a podmienky čiastočne líšiť, čo vysvetľuje potrebu využitia flexibilných šablón pre realizáciu platformy ako môžeme vidieť u dostupných riešení.

2.2 LLM služby

Ďalším z možných prístupov k zjednodušeniu a automatizácii nasadzovania aplikácií je využitie LLM modelov, ktoré sa v súčasnosti stávajú štandardom v oblasti programovania. Zjednodušenie môže spočívať napríklad vo vytvorení manifest súboru, teda konfiguračného súboru pre Kubernetes, alebo Dockerfile súboru pomocou LLM modelu namiesto programátora.

Existuje množstvo LLM nástrojov, ktoré sú špecializované na konkrétne nasadzovacie platformy alebo priamo na kontajnerizačné technológie. Tento prístup má však jeden zásadný nedostatok, nepokrýva samotnú fázu nasadenia aplikácie, ale iba tvorbu potrebných zdrojov v textovej podobe. Po vytvorení manifest súboru je stále potrebné ho správne aplikovať a inicializovať.

Napríklad po vytvorení Dockerfile je nutné zostaviť Docker obraz a uložiť ho do registra tak, aby bol dostupný napríklad pre Kubernetes, čo opäť vyžaduje manuálny zásah. Žiadna z týchto služieb teda neposkytuje plne automatizované nasadenie podľa definície uvedenej v jednej z predchádzajúcich kapitol tejto práce.

Medzi takéto nástroje patria napríklad:

- **Workik AI**: https://workik.com,
- **kubectl-ai**: https://github.com/GoogleCloudPlatform/kubectl-ai,
- **kube-copilot**: https://github.com/feiskyer/kube-copilot.

Taktiež je potrebné podotknúť, že väčšina týchto služieb nie je bezplatná a ich používanie je spojené s finančnými nákladmi, vrátane nákladov na prevádzku LLM modelov. Momentálnym trendom je vývoj takzvaných kopilotov, ktorý sú schopný aj interakcie s daným prostredím, teda môžu vykonávať príkazy a podobne. Avšak títo kopiloti sú len CLI, Rozhranie Príkazového Riadku, z angl. *Command Line Interface* aplikácie, ktoré musia byť explicitne spustené používateľovým dopytom v určitom tvare a sú veľmi všeobecné.

2.3 Zhodnotenie aktuálneho výskumu danej problematiky

3 Motivácia a ciele práce

Za touto prácou stoja osobné skúsenosti získané počas pôsobenia v korporátnom prostredí na pozícii operačného inžiniera. V takomto prostredí sa inžinieri takmer každý deň stretávajú s požiadavkami na nasadenie nových projektov do produkcie. Tieto požiadavky sú však vždy takmer rovnaké alebo len mierne odlišné. Táto skutočnosť vedie k potrebe vykonávať množstvo opakujúcich sa a málo zaujímavých úloh, ktoré má inžinier zvládnuté už mnohokrát. Napriek tomu, že ide o repetitívnu a na prvý pohľad nezáživnú prácu, má tento charakter aj svoje výhody. Práve úlohy, ktoré sú opakujúce sa a rutinné, sú vo väčšine prípadov vhodnými kandidátmi na automatizáciu.

V konkrétnom prípade sa ako nástroje na nasadzovanie používajú technológie Docker a Kubernetes, spolu s platformou na verzionovanie zdrojového kódu. Cieľom takého projektu, ktorý je odovzdaný operačnému inžinierovi, je vytvorenie CI/CD, ktoré zabezpečuje automatizované pre-nasadenie aplikácie po vykonaní určitých zmien v kóde. Avšak ešte predtým, ako môže operačný inžinier začať s tvorbou samotnej CI/CD, je potrebné pripraviť Dockerfile a manifestačné súbory pre Kubernetes. Tieto súbory je následne nutné otestovať a zabezpečiť ich automatické spúšťanie prostredníctvom CI/CD nástrojov. Uvedené kroky sú vo veľkej miere repetitívne, a preto aj náchylné na chyby spôsobené nepozornosťou. Takéto chyby môžu inžiniera stáť značné množstvo času a v konečnom dôsledku aj finančných prostriedkov.

Cieľom tejto práce je vytvorenie aplikácie alebo platformy, ktorá umožní nasadenie aplikácie do produkčného prostredia výhradne na základe poskytnutého zdrojového kódu obsahujúceho Python aplikáciu a to s takmer minimálnym zásahom zo strany programátora. Minimálnym zásahom sa rozumie iba stručné doplnenie potrebných metadát v textovej podobe pred začatím procesu. Podmienkami na splnenie stanovených cieľov sú, plne automatizované nasadenie aplikácie bez zásahu programátora pre ľubovoľný framework alebo knižnicu z vybraných podporovaných typov aplikácií, ako aj výrazná redukcia finančných nákladov spojených s nasadzovaním a prevádzkou.

Navrhovaná platforma danú úlohu realizuje s využitím generatívnej umelej inteligencie, konkrétne veľkých jazykových modelov. Tieto modely sú použité na vytvorenie takzvaného agenta, ktorého koncept je bližšie definovaný v nasledujúcej kapitole. Úlohou tohto agenta je generovanie potrebného kódu na vytvorenie súborov nevyhnutných pre nasadenie aplikácie na základe vstupného zdrojového kódu. Agent má zároveň prístup k infraštruktúre, ktorú bude môcť aktívne využívať na vykonanie

samotného nasadenia, ako aj k nástrojom určeným na testovanie navrhovaného riešenia.

Je dôležité podotknúť, že táto aplikácia nevystupuje v úlohe správcu infraštruktúry, ale iba jej používateľa. Vhodným prirovnaním je porovnanie uvedeného agenta s operačným inžinierom.

Na overenie konceptu práce boli zvolené prístupy a metódy podrobne popísané v nasledujúcich kapitolách. Vrátane nie priamo agentického riešenia.

4 Návrhy riešenia pomocou generatívnej umelej inteligencie

Táto práca navrhuje riešenie automatického nasadenia do produkcie pomocou GenAI, respektíve veľkých jazykových modelov, ktoré slúžia ako principiálna časť takzvaných umelo-inteligenčných agentov alebo skrátene agentov. Umelo-inteligenčný agent je podľa jednej z mnohých definícii definovaný ako, „systém rozhodujúci o akciách a stavoch aplikácie na základne výstupov z veľkých jazykových modelov [28]“.

Cieľom minimovej práce je teda overenie hypotézy, podľa ktorej by vyššie definovaný agent mal byť schopný nahradiť operačného inžiniera pri nasadzovaní aplikácie do produkčného prostredia. Produkčným prostredím sa rozumie Kubernetes klaster. Teda pred nasadením musí byť vytvorený Dockerfile z ktorého sa následne vygeneruje Docker obraz, ktorý je použitý v Kubernetes klastery.

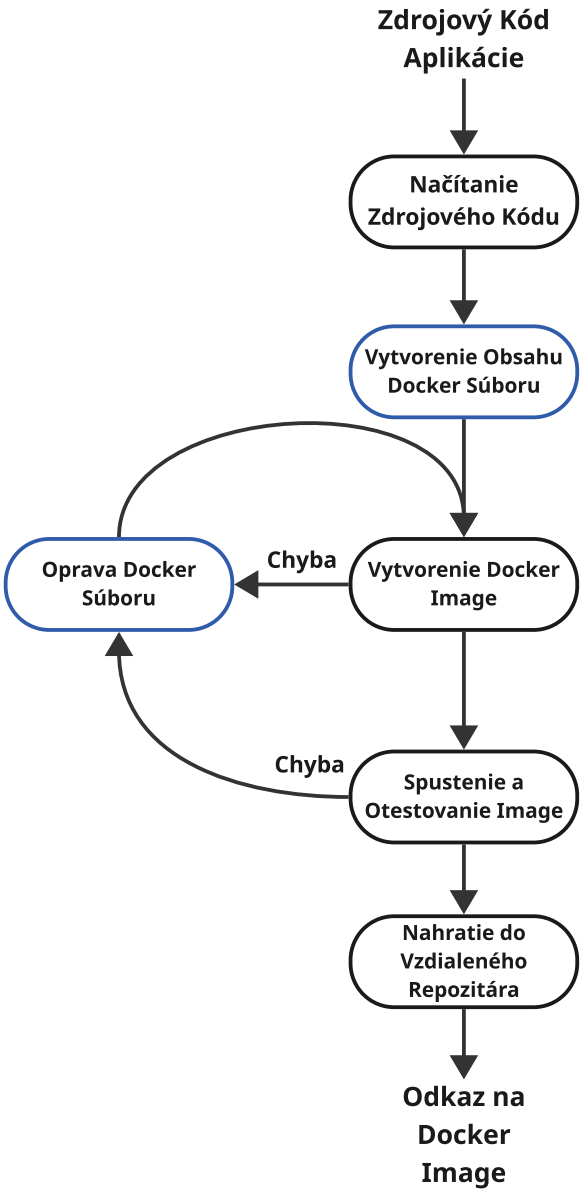
Táto práca sa pre overenie tejto hypotézy bude zaoberať výlučne Docker časťou, keďže rovnaký prístup by malo byť možné aplikovať aj na Kubernetes časť.

4.1 Riadený proces využívajúci LLM

Pod pojmom riadený proces využívajúci veľké jazykové modely (LLM) sa rozumie architektúra s nízkou mierou agentickosti, v ktorej je postup riešenia striktne vopred definovaný. LLM v tomto prípade nevystupuje ako autonómny agent, ktorý by dynamicky rozhodoval o ďalších krokoch procesu, ale plní úlohu inteligentného transformačného komponentu vloženého do pevného algoritmického rámca.

Ako už bolo uvedené, jednotlivé kroky procesu sú deterministické a pevne určené návrhom systému. LLM má vplyv predovšetkým na kvalitu výstupu, nie však na samotnú štruktúru toku riadenia. Proces je lineárny, avšak obsahuje riadené spätné väzby, ktoré umožňujú korekciu chýb bez nutnosti manuálneho zásahu používateľa. Tento prístup bol do práce zaradený ako porovnávacia vzorka voči agentickým riešeniam, ktoré sa vyznačujú vyššou mierou autonómie, adaptivity a rozhodovacej slobody.

Na nasledujúcej vizualizácií 7 je možné vidieť návrh riadeného procesu, kde jednotlivé časti využívajúce LLM a teda jeden prompt sú zvýraznené modrou farbou.



Obr. 7: Vizualizácia návrhu riadeného procesu využívajúceho LLM

Zdrojový kód aplikácie

Proces začína vstupom vo forme existujúceho zdrojového kódu aplikácie. Tento kód predstavuje jediný primárny vstup z ktorého systém odvodzuje všetky ďalšie artefakty. Predpokladá sa, že ide o funkčný projekt, avšak bez explicitne definovanej kontajnerizačnej konfigurácie.

Načítanie zdrojového kódu

V tomto kroku dochádza k analýze a extrakcii relevantných informácií zo zdrojového

kódu. Tento krok slúži ako príprava dát pre následnú prácu LLM a nemá agentický charakter.

Vytvorenie obsahu Dockerfile(LLM krok)
Tento krok predstavuje hlavné miesto integrácie LLM v procese a je vo vizualizácii zvýraznený modrou farbou. Na základe výstupov z predchádzajúcej analýzy je LLM použitý na:

- návrh vhodného Dockerfile,
- definovanie krokov potrebných na zostavenie aplikácie,
- určenie spôsobu spustenia aplikácie v kontajneri.

LLM tu disponuje ako expertný systém zakódovaný do jedného promptu, pričom nevykonáva žiadne rozhodovanie o ďalšom postupe procesu. Kvalita tohto kroku má zásadný vplyv na úspešnosť všetkých nasledujúcich fáz.

Vytvorenie Docker obrazu
Na základe vygenerovaného Dockerfile je iniciovaný proces zostavenia Docker obrazu. Tento krok je plne deterministický a vykonávaný externým nástrojom.

V prípade úspechu proces pokračuje ďalej, v opačnom prípade je identifikovaná chyba a aktivuje sa spätná väzba.

Oprava Dockerfile(LLM krok)
Ak počas vytvárania Docker obrazu dôjde k chybe, proces sa presmeruje do korekčnej vetvy. V tomto kroku je LLM opätovne využitý na:

- interpretáciu chybového výstupu,
- identifikáciu pravdepodobnej príčiny zlyhania,
- návrh úpravy Dockerfile.

Aj v tomto prípade ide o ne-agentické využitie LLM, keďže rozhodnutie o návrate do tohto kroku je striktne podmienené detekciou chyby. Po oprave sa proces opätovne vracia k vytvoreniu Docker obrazu, čím vzniká uzavretá korekčná slučka.

Spustenie a otestovanie obrazu
Po úspešnom zostavení Docker obrazu nasleduje jeho spustenie a vykonanie základných testov. Cieľom tohto kroku je overiť:

- schopnosť Docker obrazu korektne sa spustiť,
- dostupnosť hlavnej funkcionality aplikácie,
- absenciu kritických chýb.

V prípade zlyhania testov sa proces opäť vracia do kroku opravy Docker súboru, čím sa zabezpečuje minimálna úroveň kvality výstupu.

Nahratie do vzdialeného repozitára
Po úspešnom otestovaní je Docker obraz nahraný do vzdialeného repozitára. Tento krok predstavuje záver automatizového procesu a je vykonaný bez účasti LLM.

Výstup, odkaz na Docker obraz
Výsledkom celého procesu je jednoznačný odkaz na vytvorený Docker obraz, ktorý je možné ďalej distribuovať, nasadiť alebo integrovať do CI/CD prostredia.

Zhrnutie návrhu procesu
Navrhnutý riadený proces demonštruje, že LLM je možné efektívne využiť aj v neagentickom režime, kde plní úlohu znalostného modulu integrovaného do klasického automatizačného procesu. Vďaka integrácií LLM je možné vynechať potrebu operačného inžiniera alebo výber z pre-definovaných typov aplikácií. Takýto prístup ponúka:

- vysokú mieru predvídateľnosti,
- jednoduchšiu validáciu správania systému,
- nižšie riziko nežiaducej autonómie.

Zároveň však poukazuje na limity tohto prístupu, najmä jeho závislosť od kvality jednorazových výstupov LLM, čo ho robí vhodným kandidátom na porovnanie s plne agentickými architektúrami v ďalších častiach práce.

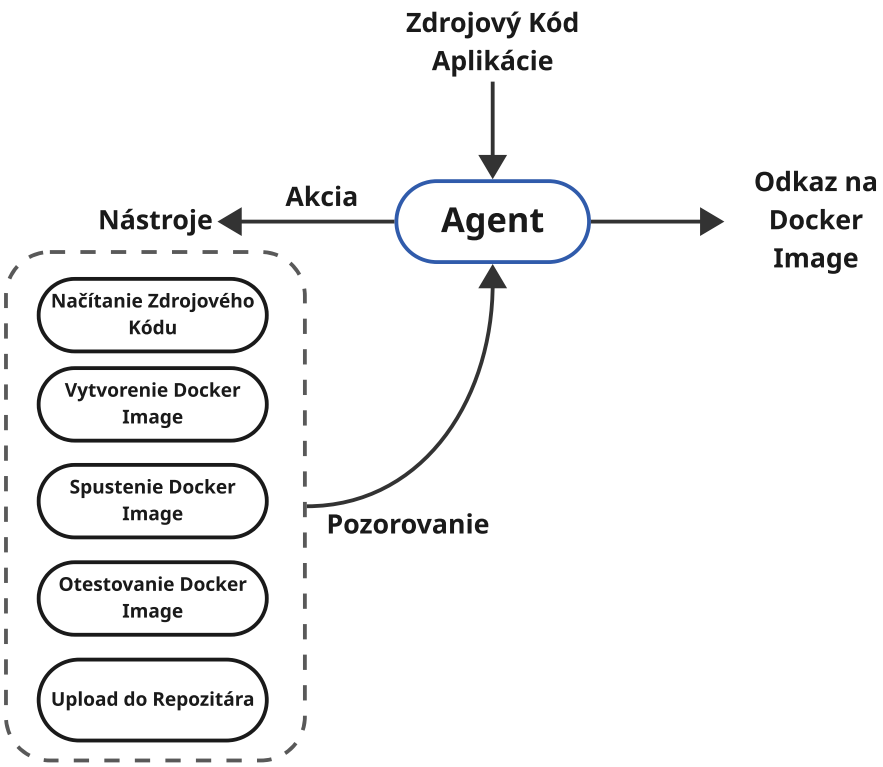
4.2 Monolitický agent

Navrhovaný prístup predstavuje monolitického softvérového agenta, ktorý využíva veľký jazykový model ako centrálny rozhodovací mechanizmus. Cieľom agenta je automatizovane, výhradne na základe zdrojového kódu aplikácie, zabezpečiť kompletný proces vytvorenia Docker obrazu, jeho otestovanie a následné nahratie do

vzdialeného repozitára. Agent tak abstrahuje celý proces kontinuálnej integrácie a prípravy artefaktu do jedného autonómneho systému.

Z hľadiska úrovne autonómnosti ide o vysoký stupeň agentickosti, keďže agent neriadi iba sekvenčné vykonávanie vopred definovaných krokov, ale aktívne rozhoduje o ďalšom postupe, vyhodnocuje výsledky predchádzajúcich akcií, dokáže proces opakovať, prerušiť alebo ukončiť a na záver vrátiť výsledok vo forme odkazu na vytvorený Docker obraz.

Na nasledujúcej vizualizácií 8 je možné vidieť návrh monolitického agenta, kde jednotlivé časti využívajúce LLM a teda jeden prompt sú zvýraznené modrou farbou



Obr. 8: Vizualizácia monolitického agenta

Zdrojový kód aplikácie

Primárnym vstupom do systému je zdrojový kód aplikácie, ktorý agent prijíma ako počiatočný kontext. Tento vstup predstavuje jediný zdroj informácií, z ktorého agent odvodzuje:

- typ aplikácie,
- potrebné závislosti,

- vstupný bod.

Agent nevychádza z explicitných inštrukcií používateľa o tom, ako má Docker obraz vyzerať, ale tieto rozhodnutia robí autonómne na základe analýzy zdrojového kódu.

Agent
Ako je znázornené na obrázku 8, agent tvorí centrálny uzol architektúry, pričom všetky vstupy, výstupy aj interakcie s nástrojmi sú sprostredkované práve cez neho. Vnútorná logika agenta je riadená jedným veľkým jazykovým modelom, ktorý pracuje s jednotným promptom a kontextom. Tento model je zodpovedný za:

- interpretáciu vstupného zdrojového kódu aplikácie,
- výber vhodných nástrojov v danom momente,
- rozhodovanie o poradí jednotlivých krokov,
- vyhodnocovanie výsledkov vykonaných akcií,
- rozhodnutie o ukončení procesu.

Použitie jedného LLM modelu v rámci celého agenta zodpovedá monolitickému prístupu, kde neexistuje explicitné delenie na vnorených agentov alebo samostatné rozhodovacie jednotky.

Nástroje
Agent má k dispozícii množinu externých nástrojov, ktoré predstavujú jeho akčný priestor. Tieto nástroje sú špeciálne prispôsobené tak, aby ich bolo možné volať prostredníctvom veľkého jazykového modelu. Ako znázorňuje ľavá časť obrázka, medzi tieto nástroje patria najmä:

- načítanie a analýza zdrojového kódu,
- vytvorenie Docker obrazu,
- spustenie Docker obrazu,
- testovanie vytvoreného kontajnera,
- nahratie obrazu do repozitára.

Agent nepracuje priamo s nízko-úrovňovými príkazmi, ale inštruuje nástroje na vykonanie konkrétnych akcií. Samotné vykonanie operácie je oddelené od rozhodovania, čo zvyšuje modularitu a bezpečnosť riešenia.

Pozorovanie a spätná väzba

Dôležitým prvkom architektúry je mechanizmus pozorovania. Po každej vykonanej akcii nástroja je výsledok, či už úspešný alebo neúspešný, vrátený späť agentovi vo forme štruktúrovanej spätnej väzby. Na základe tejto spätnej väzby agent vyhodnocuje, či bol krok úspešný, rozhoduje o potrebe opakovania alebo úpravy postupu a prípadne pokračuje ďalším krokom procesu.

Tento cyklus, akcia – pozorovanie – rozhodnutie sa opakuje až do momentu, keď agent dospeje k záveru, že cieľ bol splnený alebo že proces nie je možné úspešne dokončiť.

Ukončenie procesu a výstup

Výstupom činnosti agenta je odkaz na vytvorený Docker obraz uložený v repozitári, ako je znázornené na pravej strane obrázka. Agent sám rozhoduje o ukončení procesu, pričom toto rozhodnutie je výsledkom interného vyhodnotenia všetkých predchádzajúcich krokov.

Takýto prístup umožňuje:

- minimalizovať zásah používateľa do procesu,
- zvýšiť mieru automatizácie,
- centralizovať logiku riadenia do jedného inteligentného komponentu.

Zhrnutie návrhu procesu

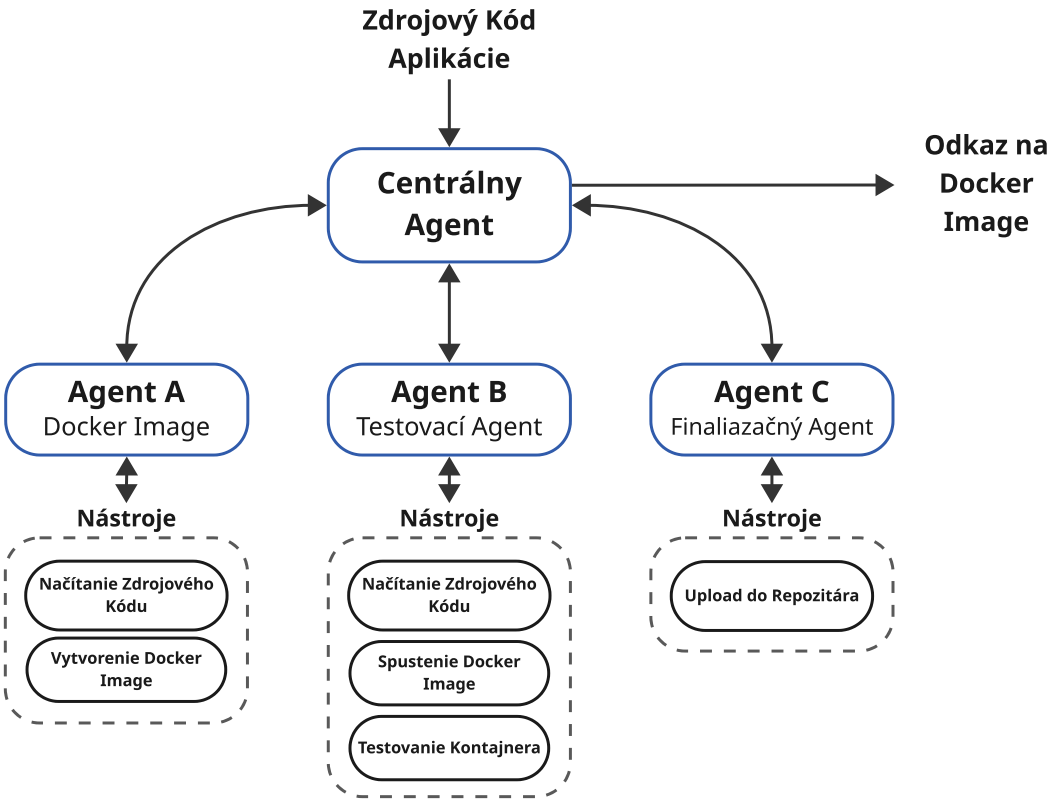
Navrhovaný monolitický agent predstavuje kompaktný, autonómny systém, v ktorom veľký jazykový model zohráva úlohu centrálneho plánovača aj rozhodovacej jednotky. V porovnaní s tradičnými riadenými procesmi ide o konceptuálne odlišný prístup, kde sa sekvencia krokov neurčuje vopred, ale vzniká dynamicky na základe priebežného vyhodnocovania stavu systému.

Tento návrh je vhodný najmä pre scenáre, kde je potrebná vysoká flexibilita, adaptabilita a schopnosť pracovať s neúplnými alebo heterogénnymi vstupmi, čo je typické práve pre automatizovanú kontajnerizáciu rôznorodých aplikácií.

4.3 Multi-agentický systém využívajúci vnorených agentov

Alternatívou k monolitickému agentickému prístupu je návrh Multi-agentického systému, v ktorom je zodpovednosť za jednotlivé fázy procesu explicitne rozdelená medzi viacero špecializovaných agentov. Tento prístup vychádza z princípov dekompozície úloh a modulárneho riadenia, pričom každý agent disponuje vlastným promptom, kontextom a obmedzeným akčným priestorom.

Ako znázorňuje obrázok 9, architektúra pozostáva z centrálneho riadiaceho agenta, ktorý koordinuje činnosť viacerých vnorených agentov. Centrálna jednotka neimplementuje samotnú logiku tvorby Docker obrazu, ale plní úlohu orchestrátora, ktorý rozhoduje o tom, ktorý špecializovaný agent má byť v danom momente aktivovaný.



Obr. 9: Vizualizácia Multi-agentického Systému

Úloha centrálneho agenta

Centrálny agent predstavuje najvyššiu úroveň rozhodovania v systéme. Jeho hlavnou úlohou je:

- prijatie vstupného zdrojového kódu aplikácie,
- rozdelenie celého procesu na logické fázy,
- delegovanie jednotlivých fáz na príslušných vnorených agentov,
- vyhodnocovanie ich výstupov a rozhodovanie o ďalšom postupe,
- ukončenie procesu a vrátenie finálneho výsledku.

Na rozdiel od monolitického prístupu centrálna jednotka nevykonáva jednotlivé kroky priamo, ale pracuje výhradne s abstraktnými výsledkami podriadených agentov. Tento model znižuje komplexitu centrálneho promptu a umožňuje lepšiu kontrolu nad správaním systému.

Agent A: Docker obraz
Prvým špecializovaným komponentom je Agent A, ktorého zodpovednosťou je fáza tvorby Docker obrazu. Tento agent pracuje so zdrojovým kódom aplikácie a disponuje nástrojmi potrebnými na jeho analýzu a následnú kontajnerizáciu. Agent A sa sústreďuje výlučne na:

- porozumenie danej štruktúry a charakteru aplikácie,
- návrh vhodného spôsobu kontajnerizácie,
- vytvorenie Docker obrazu na základe dostupného kódu.

Výsledkom jeho činnosti nie je finálny artefakt určený na distribúciu, ale intermediárny Docker obraz, ktorý je pripravený na ďalšie overenie. Tento výstup je následne odovzdaný centrálnemu agentovi.

Agent B: Testovací agent
Druhou fázou procesu je overenie funkčnosti vytvoreného obrazu, za ktorú zodpovedá Agent B teda testovací agent. Tento agent by mal byť aktivovaný až po úspešnom dokončení fázy tvorby Docker obrazu a pracuje s výstupom Agenta A.

Jeho úlohou je zabezpečiť, že Docker obraz:

- Je možné úspešne spustiť,
- vykazuje očakávané správanie aplikácie,
- spĺňa základné funkčné alebo integračné požiadavky.

Agent B disponuje vlastnými nástrojmi na spustenie kontajnera a jeho testovanie. Výsledkom jeho činnosti je hodnotenie úspešnosti testov, ktoré slúži ako rozhodovací vstup pre centrálnu jednotku. V prípade neúspechu môže centrálny agent iniciovať opakovanie predchádzajúcich krokov alebo proces ukončiť.

Agent C: Finalizačný agent

Posledným špecializovaným komponentom je Agent C, ktorý je zodpovedný za finalizačnú fázu procesu. Tento agent by mal byť aktivovaný až vtedy, keď sú predchádzajúce kroky vyhodnotené ako úspešné.

Jeho jedinou úlohou je:

- nahratie overeného Docker obrazu do vzdialeného repozitára,
- vygenerovanie odkazu na uložený image ako finálneho výstupu systému.

Agent C má úzko definovaný akčný priestor, čo minimalizuje riziko nežiaduceho správania a zároveň zvyšuje predvídateľnosť celého procesu.

Interakcia agentov a tok informácií

Komunikácia medzi agentmi prebieha nepriamo prostredníctvom centrálneho agenta, ktorý funguje ako jednotný komunikačný bod. Vnorení agenti medzi sebou nekomunikujú priamo, čím sa zachováva jasná hierarchia a riadený tok informácií.

Každý agent pracuje v rámci vlastného kontextu a promptu, čo umožňuje:

- lepšiu špecializáciu jednotlivých agentov,
- jednoduchšie ladenie správania systému,
- zvýšenú škálovateľnosť architektúry.

Zhrnutie návrhu procesu

Multi-agentický systém s vnorenými agentmi predstavuje modulárnejší a štruktúrovanejší prístup k automatizovanej tvorbe Docker obrazu v porovnaní s monolitickým agentom. Rozdelenie zodpovednosti medzi viacerých agentov znižuje kognitívne zaťaženie jednotlivých LLM komponentov a umožňuje presnejšie riadenie jednotlivých fáz procesu.

Tento prístup je vhodný najmä v prípadoch, kde je potrebná vyššia kontrola nad správaním systému, lepšia interpretovateľnosť rozhodnutí a možnosť nezávislej úpravy alebo rozšírenia jednotlivých častí procesu bez zásahu do celej architektúry.

5 Overenie konceptu

Všetky metodiky navrhované v tejto práci vychádzajú výlučne z teoretických poznatkov a jedná sa teda o hypotézy ktoré by na teoretickej úrovni mali fungovať. Cieľom tejto kapitoly je overiť toto tvrdenie pomocou štrukturovaných a objektívnych experimentov. Na základe vyhodnotenia týchto experimentov je možné uvážiť následne pokračovanie riešenia problematiky dizertačnou prácou.

5.1 Experimentálna metodológia

Cieľom tejto kapitoly je podrobne popísať experimentálnu metodológiu použitú na overenie a porovnanie troch architektonicky odlišných prístupov k automatizovanej tvorbe Docker obrazuu využívajúcich veľké jazykové modely. Navrhnutá metodológia je zameraná na objektívne vyhodnotenie praktickej použiteľnosti jednotlivých riešení, pričom osobitný dôraz je kladený na presnosť výsledku, nákladovosť a jednoduchosť použitia z pohľadu koncového používateľa.

5.1.1 Cieľ experimentov

Primárnym cieľom experimentov je overiť koncept generácie Docker obrazu výlučne pomocou aplikácií využívajúcich LLM, bez minimálneho zásahu používateľa a následne po splnení predpokladov implikovať túto skutočnosť aj na celkový proces nasadenia do produkcie.

Medzi vybrané metodológie pomocou ktorých bude koncept overený patria:

- riadený proces využívajúci LLM,
- monolitický agent s vysokou mierou autonómie,
- Multi-agentický systém s vnorenými agentmi.

V prípade úspešnosti viacerých z metodológií, metodológie budú medzi sebou porovnané za cieľom výberu tej najvhodnejšej. Teda experimentálna metodológia je zostavené tak aby bolo možné zodpovedať otázku, do akej miery vyššia agentickosť systému prispieva k lepšej presnosti a robustnosti riešenia, a aké kompromisy prináša z hľadiska času a finančných nákladov.

5.1.2 Testovacie vstupy

Experimenty sú navrhnuté s dôrazom na maximálne zjednodušenie interakcie používateľa so systémom. Používateľ vykonáva jediný manuálny zásah na začiatku procesu, keď poskytne cestu k zdrojovému kódu Python aplikácie,

Všetky nasledujúce kroky prebiehajú autonómne bez ďalšieho zásahu používateľa. Tento aspekt je kľúčový pre hodnotenie praktickej použiteľnosti riešení, keďže cieľom je minimalizovať kognitívne zaťaženie používateľa a požiadavky na jeho technické znalosti.

Na experimentálne overenie bola navrhnutá sada šiestich testovacích Python aplikácií, ktoré reprezentujú rôznu mieru zložitosti a jednoznačnosti. Tieto aplikácie je možné vidieť definované v tabuľke 3 a ich testovací účel v tabuľke 4. Aplikácie pokrývajú celý rozsah typických scenárov, od triviálnych skriptov až po problematické prípady s neúplnou konfiguráciou.

Takto zvolená množina vstupov umožňuje testovať nielen výkon systému v ideálnych podmienkach, ale aj jeho robustnosť, adaptabilitu a schopnosť riešiť neštandardné situácie. Výber aplikácií je zámerný a priamo podporuje ciele experimentálneho porovnania.

V tabuľke 3 je možné vidieť definíciu 6 druhov Python aplikácií, ktoré sú použité pri overovaní konceptu a experimentovaní.

Tabuľka 3: Prehľad testovacích Python aplikácií použitých v experimentoch

ID	Typ aplikácie	Charakteristika
A1	Jednoduchá skriptová aplikácia	Jednosúborová aplikácia bez externých závislostí
A2	Aplikácia so závislosťami	Aplikácia s explicitne definovanými knižnicami
A3	Modulárna aplikácia	Viacsúborová štruktúra s balíkmi
A4	Serverová aplikácia	Dlhodobo bežiaci proces s exponovaným rozhraním
A5	Konfigurovateľná aplikácia	Aplikácia závislá od externých konfiguračných vstupov
A6	Problémová aplikácia	Neúplná alebo nejednoznačná konfigurácia

V nasledujúcej tabuľke 4 je možné vidieť definovaný účel každého z testov.

Zdrojové kódy testovaných aplikácií je možné nájsť vo vzdialenom Github repozitári[Odkaz].

5.1.3 Postup experimentov

Každá testovacia aplikácia je spracovaná všetkými troma metodológiami, pričom každý experimentálny beh je opakovaný 5 krát. Opakovanie behov je nevyhnutné

Tabuľka 4: Prehľad testovaných aspektov pomocou jednotlivých testovacích aplikácií

ID	Testovaný aspekt systému
A1	Základná funkčnosť
A2	Spracovanie závislostí a stabilita vytvárania Docker Image
A3	Analýza projektovej štruktúry a identifikácia vstupného bodu
A4	Testovanie spustenia kontajnera a sieťovej dostupnosti
A5	Inferencia a správne nastavenie konfiguračných parametrov
A6	Robustnosť systému a schopnosť riešiť chybové stavy

vzhľadom na nedeterministickú povahu veľkých jazykových modelov, ktorej dôsledkom môžu byť rozdielne výstupy aj pri rovnakých vstupoch.

Tento prístup umožňuje analyzovať stabilitu jednotlivých metodológií, identifikovať variabilitu výsledkov a posúdiť spoľahlivosť systému v dlhodobom horizonte. Počet behov bol zvolený tak, aby poskytoval dostatočnú štatistickú významnosť pri realizovatelnej výpočtovej náročnosti, teda celkovo 90 vzoriek pre každú zvolenú architektúru.

Zvolená experimentálna metodológia je navrhnutá tak, aby poskytovala komplexný a vyvážený pohľad na porovnávané prístupy. Zaradenie riadeného procesu s nízkou mierou agentickosti slúži ako referenčný bod, voči ktorému je možné kvantifikovať prínosy autonómnych agentických systémov.

Metodológia zároveň reflektuje praktické obmedzenia reálneho nasadenia, ako sú náklady na používanie veľkých jazykových modelov a požiadavky na spoľahlivosť systému. Vďaka tomu sú výsledky experimentov nielen akademicky relevantné, ale aj prakticky využiteľné.

5.1.4 Zaznamenávané metriky

Hodnotiace metriky boli zvolené tak, aby reflektovali praktické aj technické aspekty riešenia. Najdôležitejšou metrikou je presnosť, ktorá vyjadruje mieru, do akej systém dokáže vytvoriť plne funkčný Docker orabz bez dodatočných zásahov.

Okrem presnosti sú sledované aj metriky časovej náročnosti, finačnej nákladnosti a stability procesu. Tieto metriky umožňujú identifikovať kompromisy medzi kvalitou výsledku a efektivitou riešenia.

Presnosť

Hlavná metrika, presnosť predstavuje hodnotiacu metriku vyjadrujúcu celkovú úspeš-

nosť riešenia na základe splnenia definovaných pod-úloh súvisiacich s vytvorením a overením funkčnosti Docker obrazu. Metrika je založená na binárnom hodnotení jednotlivých pod-úloh, pričom každá podúloha je vyhodnotená ako splnená alebo nesplnená a má priradený pevne stanovený počet bodov.

Formálne je presnosť definovaná nasledovne:

$$\text{Presnosť}_{\%} = \frac{\sum_{i=1}^{n} s_i \cdot w_i}{\sum_{i=1}^{n} w_i} \cdot 100 \tag{1}$$

kde:

- n označuje počet hodnotených pod-úloh,
- $s_i \in \{0, 1\}$ predstavuje binárny výsledok i-tej pod-úlohy, pričom $s_i = 1$ znamená, že podúloha bola úspešne splnená, a $s_i = 0$ znamená jej nesplnenie,
- w_i je bodová váha priradená i-tej podúlohe.

Binárne hodnotenie jednotlivých pod-úloh bolo zvolené s cieľom zabezpečiť jednoznačnosť, reprodukovateľnosť a minimalizáciu subjektivity hodnotenia. Každá podúloha reprezentuje jasne definovaný technický stav, ktorý je možné jednoznačne overiť. Jednotlivé podúlohy boli zvolené tak, aby pokrývali celý proces tvorby, nasadenia a overenia funkčnosti Docker obrazu aplikácie. Hodnotené podúlohy a ich bodové ohodnotenie sú definované nasledovne:

- **úspešné vytvorenie Docker obrazu**($w_1 = 15$ bodov),
- **úspešné spustenie Docker kontajnera**($w_2 = 20$ bodov),
- **úspešné vykonanie automatizovaných testov aplikácie**($w_3 = 30$ bodov),
- **manuálne vyhodnotenie funkčnosti aplikácie**($w_4 = 35$ bodov).

Výsledná hodnota metriky presnosti nadobúda hodnoty v intervale $\langle 0, 100 \rangle$ a vyjadruje percentuálnu mieru splnenia všetkých hodnotených pod-úloh vzhľadom na ich bodové váhy.

Časová náročnosť

Časová náročnosť predstavuje metriku zameranú na posúdenie efektívnosti hodnoteného riešenia z hľadiska celkového času vykonania procesu. Hlavným hodnotiacim kritériom je celkový čas behu, ktorý priamo ovplyvňuje použiteľnosť riešenia v praxi. Doplnkovým kritériom je čas tvorby Docker obrazu, ktorý slúži ako indikátor

efektívnosti procesu kontajnerizácie, avšak nemá dominantný vplyv na výslednú metriku.

V rámci hodnotenia sú sledované nasledujúce časové veličiny:

- celkový čas behu procesu T_{total} [s],
- čas tvorby Docker obrazu T_{build} [s].

Na vyjadrenie časovej náročnosti je definovaná bodová metrika, ktorá kombinuje obe sledované veličiny s rôznou váhou ich významu:

$$\text{ČasováNáročnosť} = 0.9 \cdot \frac{1}{T_{\text{total}}} + 0.1 \cdot \frac{1}{T_{\text{build}}} \tag{2}$$

kde vyššia hodnota časovej náročnosti indikuje nižšiu časovú náročnosť a teda efektívnejšie riešenie. Dominantný vplyv celkového času zabezpečuje, že optimalizácia času tvorby Docker obrazu nemá neprimeraný vplyv na celkové hodnotenie.

Táto metrika umožňuje priamo porovnať rôzne riešenia medzi sebou na základe nameraných časov, bez potreby externých referenčných hodnôt.

Nákladovosť

Nákladovosť riešenia je hodnotená z pohľadu efektívnosti využitia LLM a súvisiacich zdrojov. Hlavným cieľom je sledovať odhadovanú cenu všetkých API volaní LLM počas jedného kompletného behu riešenia, ktorá priamo reflektuje ekonomickú náročnosť.

Pre doplnkové posúdenie sa sledujú aj:

- Počet volaní LLM(N_{calls}) – predstavuje, koľkokrát je model počas jedného behu zavolaný. Vyšší počet volaní môže zvyšovať latenciu a zložitosť spracovania výstupov.
- Celkový počet tokenov[7](N_{tokens}) – súčet odoslaných a prijatých tokenov počas jedného behu. Tokeny sú primárnym faktorom ovplyvňujúcim cenu.

Hlavná metrika, cena jedného spustenia experimentu(C_{run}) je byť vyjadrená ako:

$$C_{\text{run}} = \sum_{i=1}^{N_{\text{calls}}} \text{Cena volania}_i \tag{3}$$

alebo, pokiaľ je cena za token jednotná:

$$C_{\text{run}} = N_{\text{tokens}} \cdot P_{\text{token}} \tag{4}$$

kde:

[7]Základná jednotka textu spracovávaná LLM

- C_{run} je odhadovaná cena všetkých API volaní LLM počas jedného kompletného behu riešenia,
- N_{calls} je počet volaní LLM,
- N_{tokens} je celkový počet tokenov použitých počas behu,
- P_{token} je cena za jeden token podľa poskytovateľa LLM.

Takýto prístup umožňuje porovnávať rôzne riešenia primárne podľa ekonomickej efektivity, pričom počet volaní a tokenov poskytuje doplnkovú informáciu o zdrojoch, ktoré prispievajú k celkovej cene.

Stabilita

Stabilita hodnotených riešení je posudzovaná na základe výskytu zlyhaní v jednotlivých fázach procesu. Cieľom tejto metriky je identifikovať, v ktorých krokoch automatizovaného procesu dochádza k najčastejším zlyhaniam a ako sa tieto zlyhania líšia medzi navrhovanými prístupmi.

Zlyhanie je definované ako stav, v ktorom výstup poskytnutý modelom nedosahuje požadovanú úroveň funkčnosti. Napríklad pri spúšťaní, Docker obrazu nemusí byť spustený z dôvodu syntaktickej chyby. V takomto prípade by mal model zareagovať a spätnou väzbou umožniť LLM nápravu.

Stabilita je sledovaná samostatne pre nasledujúce fázy procesu:

- vytvorenie Docker obrazu – zlyhania súvisiace s generovaním Dockerfile, riešením závislostí, chybami pri zostavení obrazu alebo neúspešným vytvorením,
- spustenie Docker obrazu – zlyhania pri štarte kontajnera,
- nahranie Docker obrazu do vzdialeného repozitára – neexistujúci Docker obraz alebo nesprávne označenie.

Pre každú fázu je zaznamenávaný absolútny počet zlyhaní, ktorý je následne použitý priamo vo výsledkoch experimentov. Počet zlyhaní v j-tej fáze je definovaný nasledovne:

$$F_j = \sum_{i=1}^{k} f_{i,j} \tag{5}$$

kde:

- F_j je celkový počet zlyhaní v j-tej fáze,

- k je počet experimentálnych behov,
- $f_{i,j} \in \{0, 1\}$ označuje, či v i-tom behu došlo k zlyhaniu v j-tej fáze.

Celková stabilita riešenia je charakterizovaná vektorom:

$$\mathbf{F} = (F_{build}, F_{run}, F_{push}) \tag{6}$$

kde jednotlivé zložky reprezentujú počet zlyhaní vo fáze vytvárania Docker obrazu, spúšťania Docker obrazu a nahrávania obrazu do vzdialeného repozitára.

5.1.5 Spôsob vyhodnocovania

Pre celkové porovnanie hodnotených riešení je zavedená agregovaná hodnotiaca metrika, ktorá kombinuje všetky sledované aspekty do jedného výsledného skóre. Cieľom agregácie je umožniť jednoznačné porovnanie jednotlivých prístupov pri zachovaní rozdielnej dôležitosti hodnotených metrík.

Jednotlivým metrikám sú priradené nasledujúce váhy: **presnosť** 70 %**, nákladovosť** 20 %**, časová náročnosť** 5 % **a stabilita** 5 %**.** Takéto rozdelenie váh vychádza z predpokladu, že primárnym cieľom hodnoteného riešenia je dosiahnutie vysokej presnosti, teda schopnosti automatizovane vytvoriť a overiť plne funkčný Docker obraz bez potreby manuálnych zásahov. V prípade dosiahnutia vysokej presnosti je stabilita riešenia do veľkej miery implicitne zabezpečená, keďže zlyhania jednotlivých fáz procesu sa priamo premietajú do zníženia hodnoty presnosti.

Časová náročnosť má v rámci metodológie nižšiu váhu, nakoľko hodnotené riešenia nie sú určené pre vysoko frekventované nasadzovanie do produkčného prostredia ale skôr na poskytnutie prvotného plánu pomocou ktorého sa následne vytvorí CI/CD, kde by boli kladené prísne požiadavky na latenciu.

Nákladovosť riešenia je hodnotená samostatnou váhou 20 %, pričom sa vychádza z predpokladu, že všetky navrhované riešenia dosahujú výrazne nižšie náklady v porovnaní s existujúcimi komerčnými službami poskytujúcimi obdobnú funkcionalitu. Metrika nákladovosti preto slúži predovšetkým na vzájomné porovnanie navrhovaných prístupov medzi sebou, nie na absolútne posúdenie ich ekonomickej výhodnosti.

Pred agregáciou sú všetky metriky normalizované do intervalu $\langle 0, 1 \rangle$ s cieľom zabezpečiť ich porovnateľnosť. Presnosť, definovaná v intervale $\langle 0, 100 \rangle$, je normalizovaná nasledovne:

$$A_{norm} = \frac{A}{100} \tag{7}$$

Nákladovosť, časová náročnosť a stabilita sú minimalizačné metriky, pričom ich normalizácia je realizovaná relatívne voči najlepšiemu dosiahnutému výsledku v rámci porovnávaných riešení:

$$M_{norm} = \frac{M_{min}}{M} \tag{8}$$

kde M predstavuje hodnotu danej metriky pre hodnotené riešenie a M_{min} je najnižšia hodnota danej metriky spomedzi všetkých porovnávaných riešení. V prípade stability je hodnota M definovaná ako celkový počet zlyhaní zaznamenaných počas všetkých fáz procesu.

Výsledné agregované skóre S je vypočítané ako vážený súčet normalizovaných metrík:

$$S = 0.70 \cdot A_{norm} + 0.20 \cdot C_{norm} + 0.05 \cdot T_{norm} + 0.05 \cdot F_{norm} \tag{9}$$

Výsledná hodnota S nadobúda hodnoty v intervale $\langle 0, 1 \rangle$, pričom vyššia hodnota indikuje lepšie celkové hodnotenie riešenia.

5.2 Experimentálne prostredie

Experimentálne prostredie predstavuje technický rámec, v ktorom bolo realizované overenie navrhovaného konceptu. Jeho presná špecifikácia je nevyhnutná pre správnu interpretáciu dosiahnutých výsledkov a zabezpečenie ich reprodukovateľnosti. V tejto podkapitole sú preto opísané hardvérové a softvérové komponenty použité pri experimentoch, ako aj jazykový model, ktorý tvorí základ testovanej časti navrhovaného riešenia.

5.2.1 Hardvérové prostredie

Experimenty boli vykonané na osobnom počítači s nasledujúcou konfiguráciou:

- **procesor:** Intel Core i5-11300H(11. generácia) s taktovacou frekvenciou 3.10 GHz,
- **operačná pamäť:** 16 GB RAM,
- **úložisko:** 1 TB SSD Samsung 970 EVO Plus,
- **grafická karta:** NVIDIA GeForce RTX 3050 Laptop GPU (4 GB VRAM),
- **architektúra systému:** 64-bitový operačný systém, procesor typu x64,
- **operačný systém:** Windows 10 Home, 19045.6466.

Použitá hardvérová konfigurácia je dostatočná na lokálne spúšťanie kontajnerizovaných aplikácií a experimentovanie s jazykovými modelmi prostredníctvom externých API rozhraní.

5.2.2 Softvérové prostredie

Experimentálne prostredie je z hľadiska softvéru zámerne minimalistické a pozostáva len z komponentov nevyhnutných pre realizáciu experimentov. Použité softvérové nástroje a technológie sú uvedené v nasledujúcom zozname:

- **Docker Desktop:** verzia 4.50.0,
- **programovací jazyk Python:** verzia 3.13.

Docker Desktop slúži ako integračná vrstva medzi hostiteľským operačným systémom a kontajnerizačným prostredím. Poskytuje jednotné rozhranie na správu kontajnerov, obrazov a bežiacich služieb, ako aj podporu pre spúšťanie Docker prostredia na platforme Windows. V rámci experimentov je Docker Desktop využívaný na lokálne vytváranie Docker obrazov ako aj na testovanie aplikácií generovaných navrhovaným riešením.

Python je použitý ako implementačný jazyk testovaných aplikácií, ako aj samotného riešenia. Zvolená kombinácia technológií zabezpečuje jednotné a reprodukovateľné experimentálne prostredie.

5.2.3 Špecifikácia použitého LLM

Pre účely experimentálneho overenia bol využitý veľký jazykový model **GPT-5** od spoločnosti OpenAI.

V súčasnej fáze výskumu je experimentálne overenie realizované len s jedným jazykovým modelom. V kontexte dizertačnej práce je však vhodné rozšíriť experimenty o porovnanie viacerých jazykových modelov, aby bolo možné objektívne vyhodnotiť vplyv zvoleného modelu na kvalitu a spoľahlivosť procesu nasadzovania aplikácií do produkčného prostredia.

5.3 Experimenty a preukázanie konceptu

Pri každom z experimentov je samostatne predstavená implementácia daného riešenia spolu s použitými knižnicami a závislosťami. V poslednej podkapitole sú výsledky porovnané a vyhodnotené.

5.3.1 Riadený proces

5.3.2 Monolitický agent

5.3.3 Multi-agentický Systém

5.3.4 Porovnanie výsledkov

CLAUDE CLI Compare - Paid - Versatile / Generic vs Strctured - Save etc. maybe better

ReAct Agent zakomponovať

Záver

„ … “

Poznámka autora: ...

- Viacero teoretických odvod. - Dokončenie Kuberentes - Porovnanie Modelov

Literatúra

1. IBM. *Deploying software*. 2025. Dostupné tiež z: https://www.ibm.com/docs/en/zos/3.1.0?topic=task-deploying-software. Accessed: 2025-01-03.
2. PRESSMAN, R. S. *Software Engineering: A Practitioner's Approach*. 7. vyd. New York: McGraw-Hill, 2009. ISBN 978-007-337597-7.
3. AMAZON WEB SERVICES (AWS). *What is a Web Application?* 2025. Dostupné tiež z: https://aws.amazon.com/what-is/web-application/. Online; accessed 2026-01-04.
4. PYTHON SOFTWARE FOUNDATION. *Web Programming in Python*. Dostupné tiež z: https://wiki.python.org/moin/WebProgramming. Online; accessed 2026-01-04.
5. PALLETS PROJECTS. *Welcome to Flask*. Dostupné tiež z: https://flask.palletsprojects.com/en/stable/. Online; accessed 2026-01-04.
6. AMAZON WEB SERVICES. *What is an API?* 2025. Dostupné tiež z: https://aws.amazon.com/what-is/api/. Accessed: 2026-01-07.
7. GOOGLE CLOUD. *What is a REST API?* 2025. Dostupné tiež z: https://cloud.google.com/discover/what-is-rest-api?hl=sk. Accessed: 2026-01-07.
8. PYTHON, R. *Python and REST APIs: Interacting With Web Services* [https://realpython.com/api-integration-in-python/]. 2025. Accessed: 2026-01-08.
9. GEEKSFORGEEKS. *Python | FastAPI Introduction* [https://www.geeksforgeeks.org/python/fastapi-introduction/]. 2025. Accessed: 2026-01-08.
10. IBM. *What is a data pipeline?* [online]. International Business Machines Corporation, 2024. [cit. 2026-01-11]. Dostupné z: https://www.ibm.com/think/topics/data-pipeline.
11. IBM. *What is model training?* [online]. International Business Machines Corporation, 2024. [cit. 2026-01-11]. Dostupné z: https://www.ibm.com/think/topics/model-training.
12. PYTHON PACKAGING AUTHORITY. *Installing Packages*. 2024. Dostupné tiež z: https://packaging.python.org/installing/. Accessed: 2026-01-11.
13. PYTHON SOFTWARE FOUNDATION. *pip*. 2024. Dostupné tiež z: https://pypi.org/project/pip/. Python Package Index. Accessed: 2026-01-11.
14. ASTRAL SOFTWARE. *uv Documentation*. 2024. Dostupné tiež z: https://docs.astral.sh/uv/. Accessed: 2026-01-11.

15. POETRY DEVELOPMENT TEAM. *Poetry Documentation: pyproject.toml*. 2024. Dostupné tiež z: `https://python-poetry.org/docs/pyproject/`. Accessed: 2026-01-11.

16. GOEL, R.; BHRAMHABHATT, N. Containerization in Modern Software Deployment Pipelines. *Journal of Software Engineering and Applications*. 2024, **17**(1), 22–38.

17. MUZUMDAR, A. et al. Docker Engine Architecture and Container Runtime Internals. *International Journal of Cloud Computing*. 2024, **13**(2), 145–162.

18. CASALICCHIO, E.; IANNUCCI, S. The State of the Art in Containerization Technologies. *ACM Computing Surveys*. 2020, **52**(4), 1–36. Dostupné z DOI: `10.1145/3365870`.

19. SINGH, K. et al. Resource Management and Isolation in Linux Containers. *Journal of Systems and Software*. 2019, **156**, 123–136.

20. LARSSON, L. et al. Performance Evaluation of Containers and Virtual Machines. *Future Generation Computer Systems*. 2020, **104**, 424–438.

21. GUDELLI, V. R. Kubernetes-based orchestration for scalable cloud solutions. *International Journal of Novel Research and Development*. 2021.

22. FERREIRA, A. P.; SINNOTT, R. A performance evaluation of containers running on managed Kubernetes services. In: *Proceedings of the IEEE International Conference on Cloud Computing Technology and Science*. IEEE, 2019, s. 199–208.

23. FEUERRIEGEL, S.; HARTMANN, J.; JANIESCH, C.; ZSCHECH, P. Generative AI. *Business & Information Systems Engineering*. 2023.

24. VASWANI, A. et al. Attention Is All You Need. *Advances in Neural Information Processing Systems*. 2017, **30**.

25. KINGMA, D. P.; WELLING, M. Auto-Encoding Variational Bayes. *arXiv preprint arXiv:1312.6114*. 2013.

26. GOODFELLOW, I. et al. Generative Adversarial Nets. *Advances in Neural Information Processing Systems*. 2014, **27**.

27. CHEN, M.; TWOREK, J.; JUN, H. et al. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*. 2021.

28. CHASE, H. *What is an AI agent?* [`https://www.blog.langchain.com/what-is-an-agent/`]. 2024. Blog post on LangChain Blog, accessed 19 January 2026.

29. ZHOU, A. *What is an LLM Tool?* [`https://vercel.com/kb/guide/what-is-an-llm-tool`]. 2025. Vercel Knowledge Base.

30. DEVELOPERS, L. *Router — Multi-agent patterns* [https://docs.langchain.com/oss/python/langchain/multi-agent]. 2026. Accessed: 2026-01-22.

31. YASRAB, R. Platform-as-a-Service (PaaS): The Next Hype of Cloud Computing. *arXiv preprint arXiv:1804.10811*. 2015.

32. RENDER, INC. *Render*. 2025. Dostupné tiež z: https://render.com/. Online; accessed 13 January 2026.

33. RAILWAY CORP. *Railway*. 2025. Dostupné tiež z: https://railway.com/. Online; accessed 13 January 2026.

34. FLY.IO, INC. *Fly.io*. 2025. Dostupné tiež z: https://fly.io/. Online; accessed 13 January 2026.

35. GOOGLE LLC. *Google Cloud Run*. 2025. Dostupné tiež z: https://cloud.google.com/run. Online; accessed 13 January 2026.

Použitie nástrojov umelej inteligencie

...